Thread-Level Parallelism Coherence

ILP → TLP

instruction-level parallelism

thread-level parallelism

ILP → TLP

instruction-level parallelism

thread-level parallelism

TLP is identified by software or programmer and threads consist of (parallel) instructions

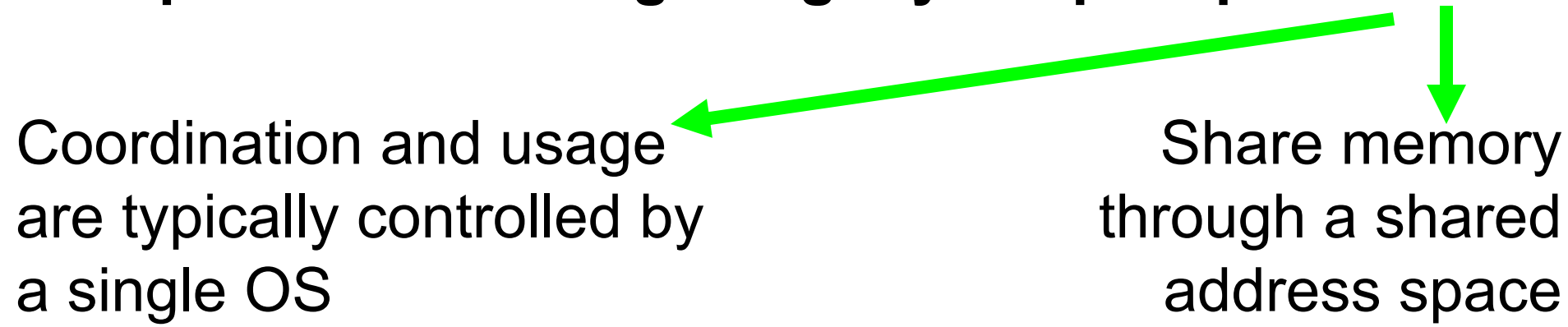
MIMD

multiple instruction streams
multiple data streams

Each processor fetches its own instructions and operates on its own data

multiprocessors

computers consisting of tightly coupled processors



multiprocessors

computers consisting of tightly coupled processors

Muticore
Single-chip systems with multiple cores

Multi-chip computers
each chip may be a multicore sys

Exploiting TLP

two software models

- **Parallel processing**

  the execution of a tightly coupled set of threads collaborating on a single task

- **Request-level parallelism**

  the execution of multiple, relatively independent processes that may originate from one or more users

Outline

- Multiprocessor Architecture
- Centralized Shared Memory
- Distributed Shared Memory

Outline

- Multiprocessor Architecture
- Centralized Shared Memory
- Distributed Shared Memory

Multiprocessor Architecture

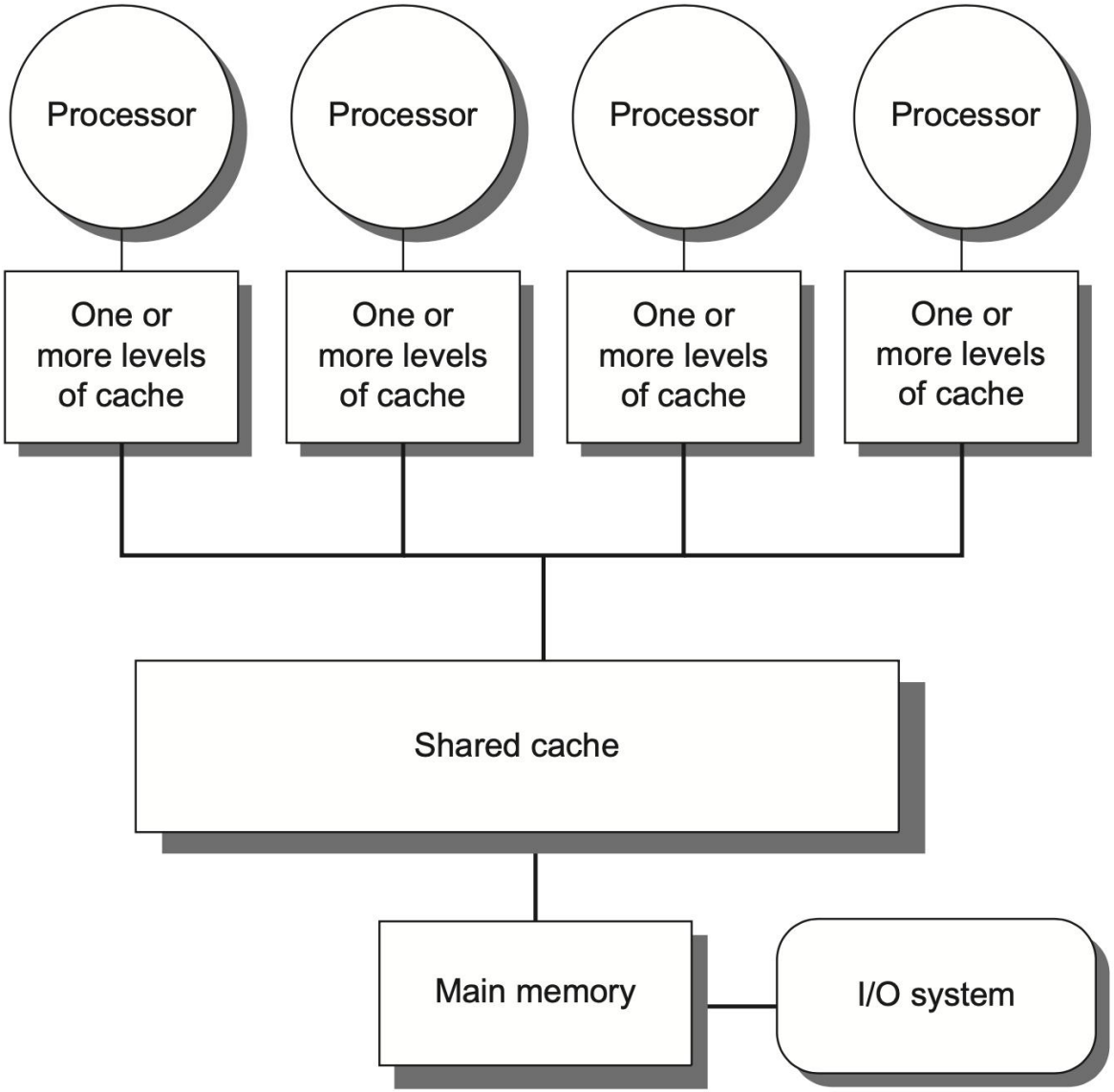
- According to memory organization and interconnect strategy
- Two classes

  symmetric/centralized shared-memory multiprocessors (SMP)

  +

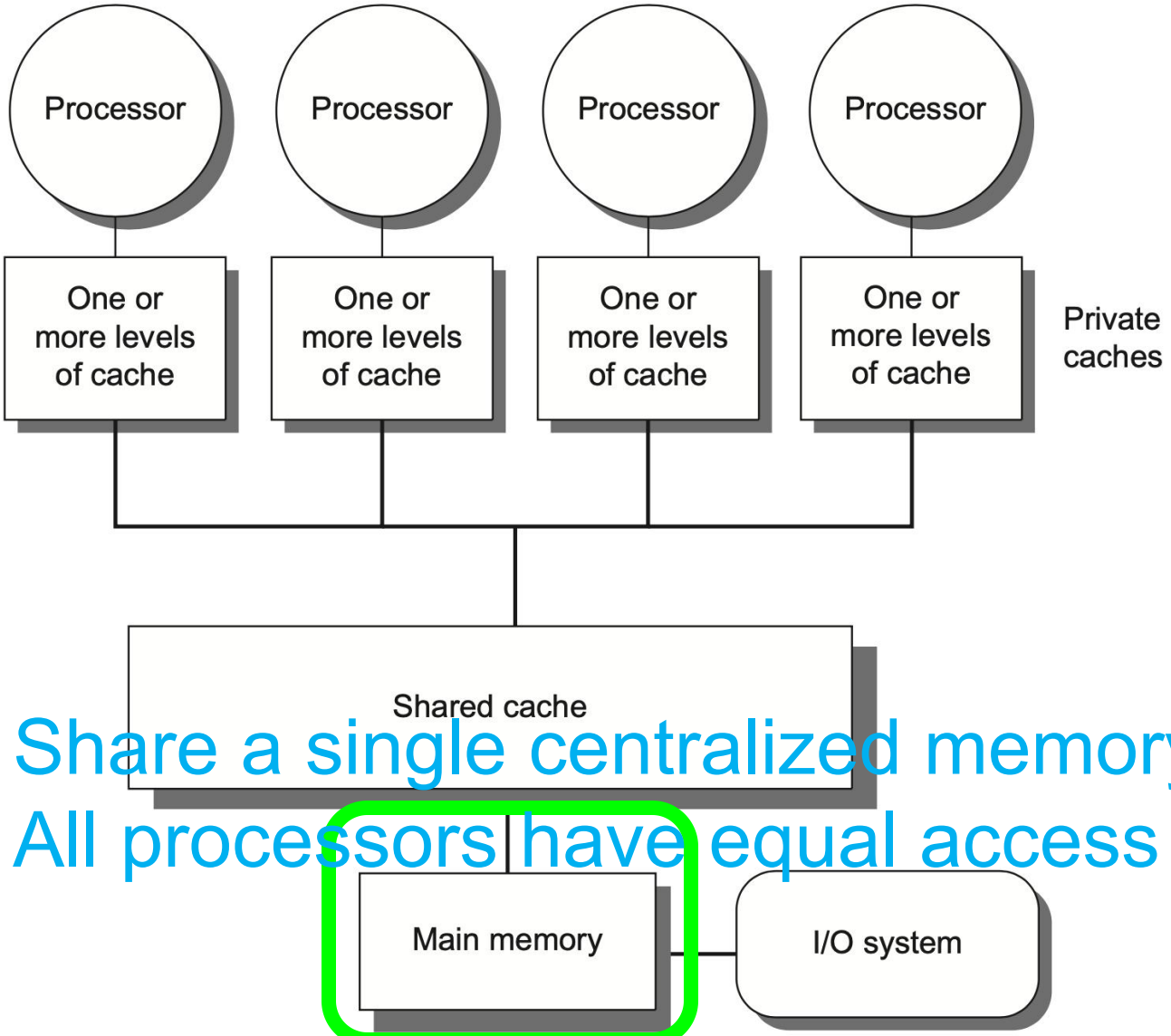
  distributed shared memory multiprocessors (DMP)

Centralized Shared-Memory



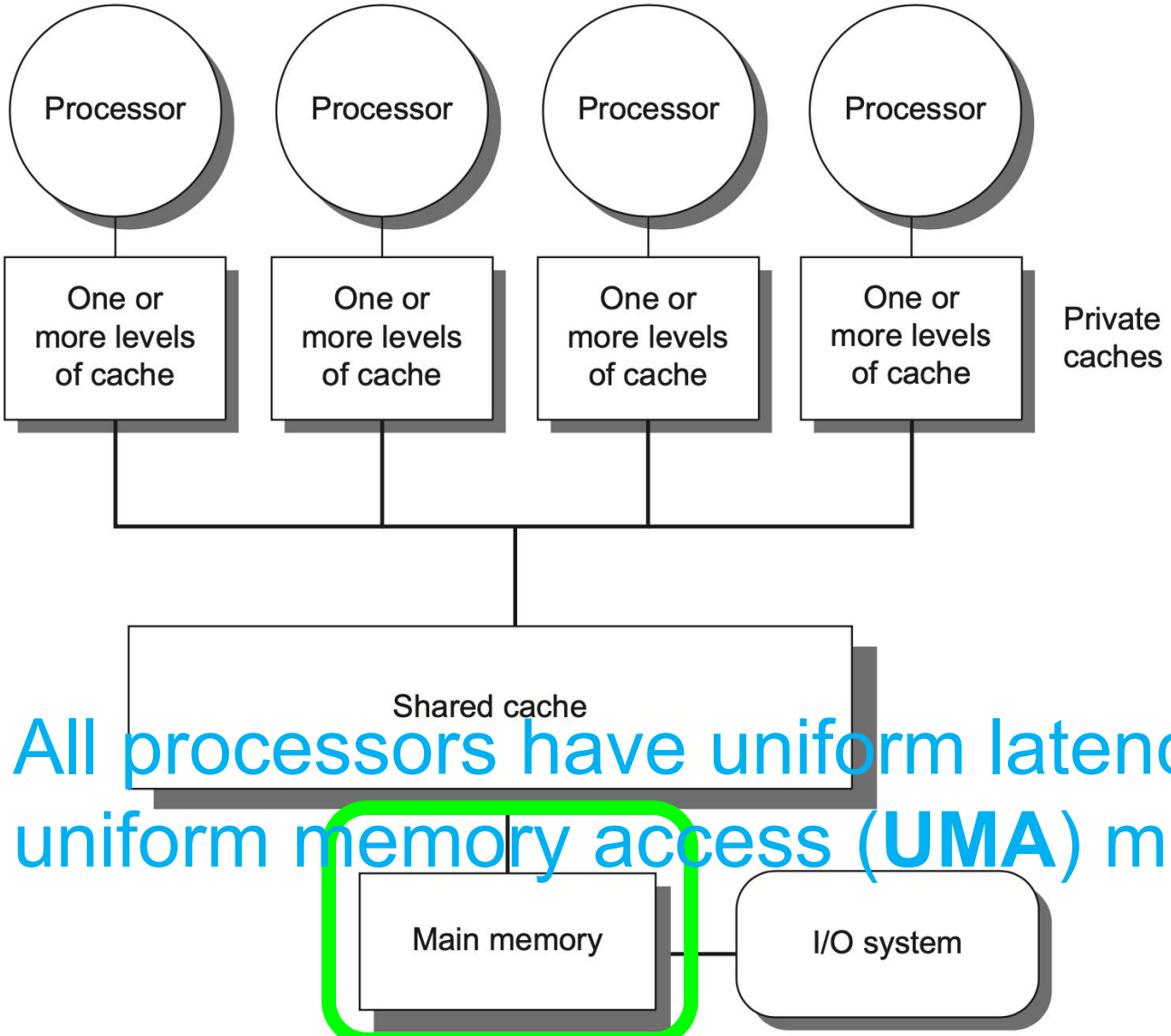
32 or fewer cores

Centralized Shared-Memory



Share a single centralized memory;
All processors have equal access

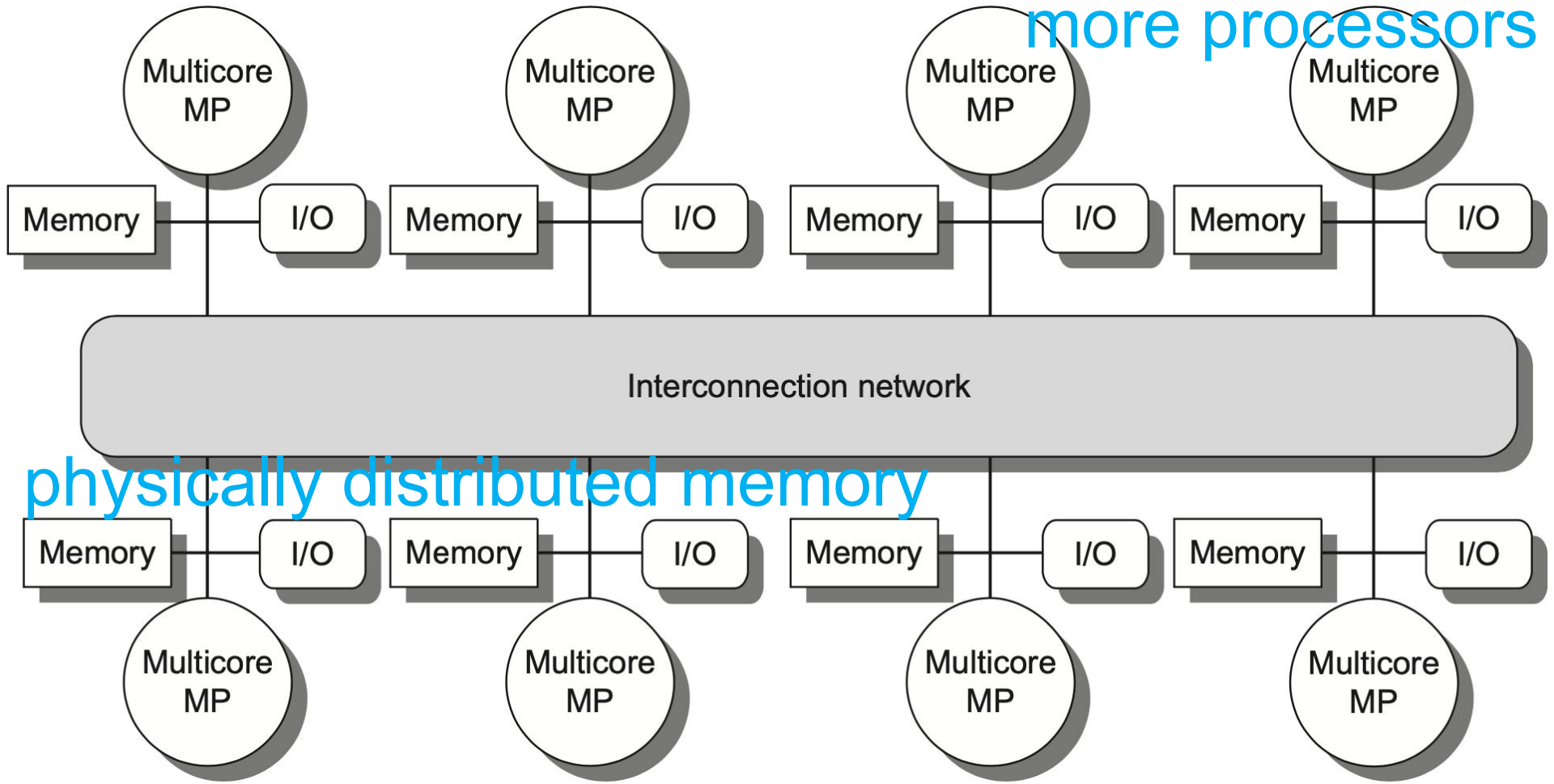
Centralized Shared-Memory



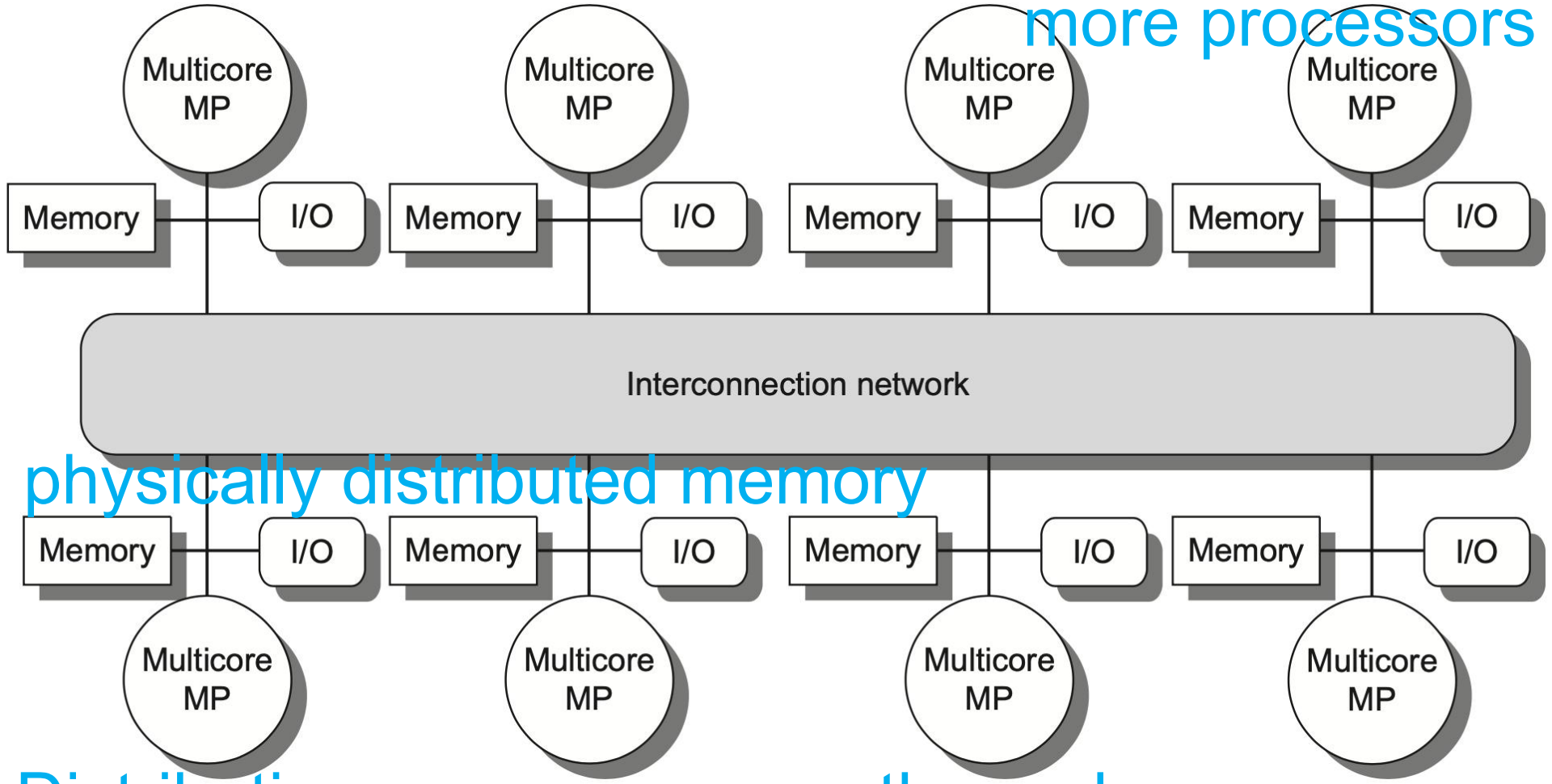
All processors have uniform latency from memory

uniform memory access (**UMA**) multiprocessors

Distributed Shared Memory



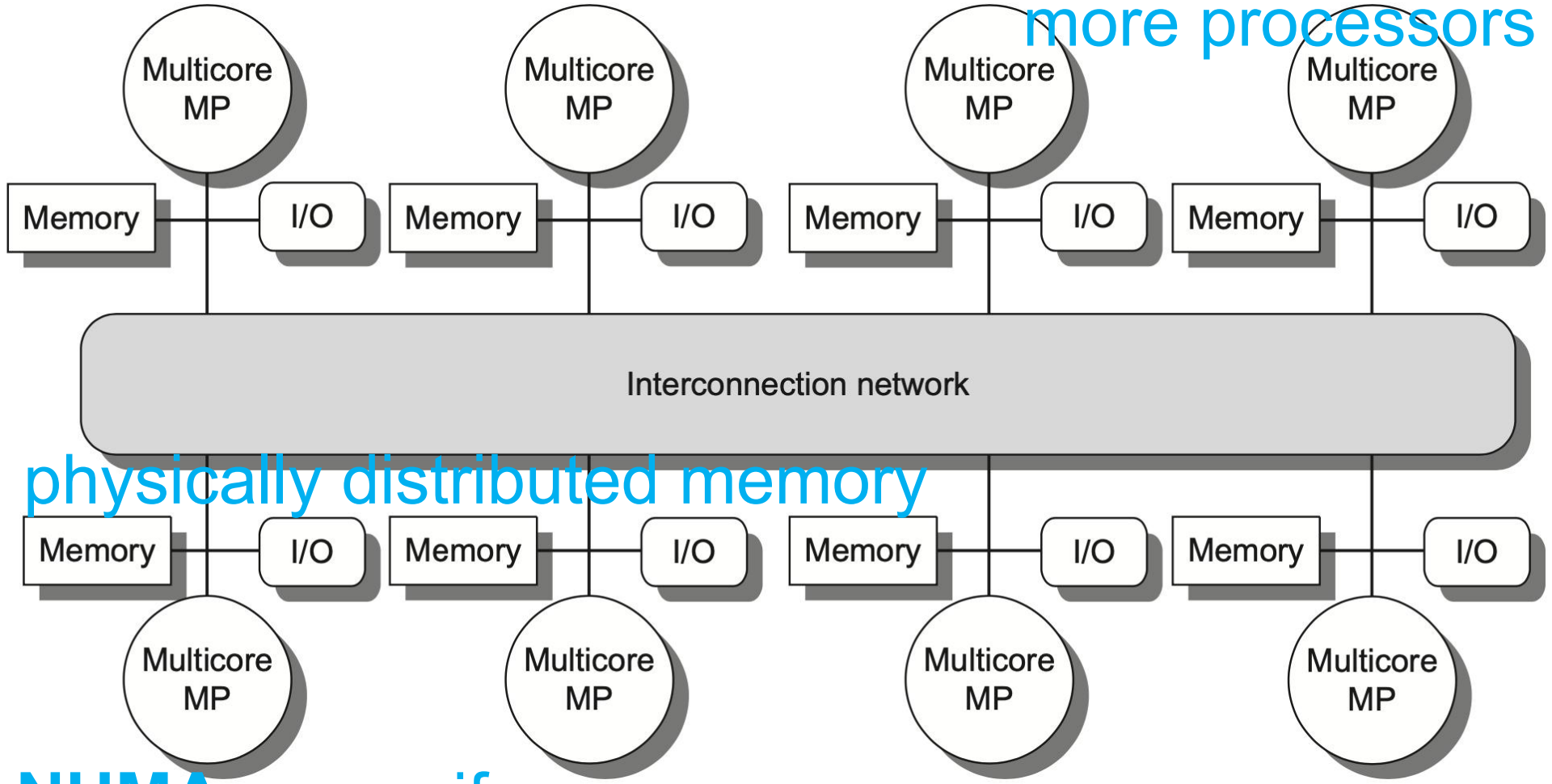
Distributed Shared Memory



Distributing memory among the nodes increases bandwidth & reduces local-mem latency

Distributed Shared Memory

more processors



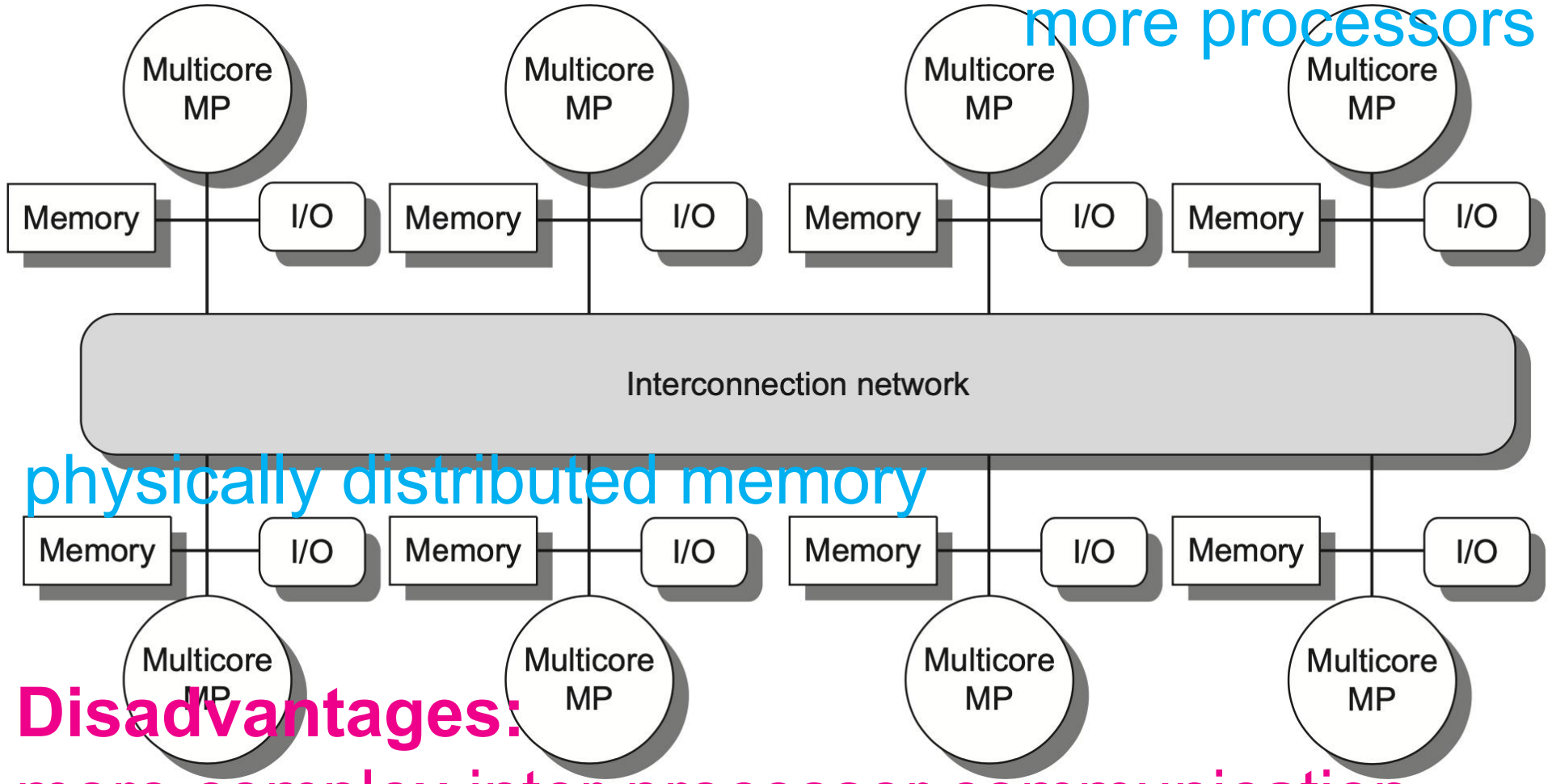
physically distributed memory

NUMA: nonuniform memory access

access time depends on data word loc in mem

Distributed Shared Memory

more processors



physically distributed memory

Disadvantages:

more complex inter-processor communication

more complex software to handle distributed mem

Hurdles of Parallel Processing

- Limited parallelism in programs
- Relatively high cost of communications

Limited Program Parallelism

- Limited parallelism in programs makes it difficult to achieve good speedups in any parallel processor

```
ld    x1, 0(x0)
ld    x2, 4(x0)
add   x3, x1, x2
add   x4, x3, 2
```

before

```
ld    x1, 0(x0)
ld    x2, 4(x0)
add   x3, x1, 1
add   x4, x2, 1
add   x5, x3, x4
```

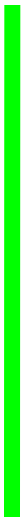
after

compute A+B+2

Limited Program Parallelism

- Limited parallelism in programs makes it difficult to achieve good speedups in any parallel processor

```
ld    x1, 0(x0)
ld    x2, 4(x0)
add   x3, x1, x2
add   x4, x3, 2
```

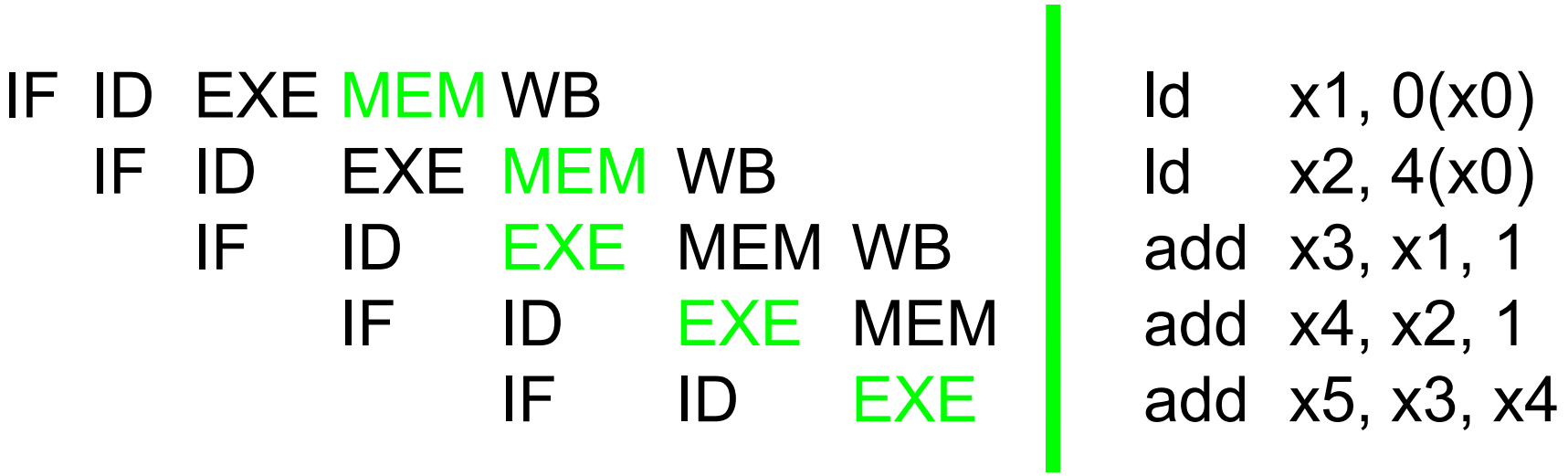


```
IF   ID   EXE   MEM   WB
     IF   ID    EXE   MEM   WB
          IF    ID    EXE   EXE
                IF    ID    EXE
```

before

Limited Program Parallelism

- Limited parallelism in programs makes it difficult to achieve good speedups in any parallel processor



Limited Program Parallelism

- Limited parallelism affects speedup
- **Example 1**

  to achieve a speedup of 80 with 100 processors, what fraction of the original computation can be sequential?

  Answer

  by Amdahl's law

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} + (1 - \text{Fraction}_{\text{enhanced}})}$$

Limited Program Parallelism

- Limited parallelism affects speedup
- **Example 1**

  to achieve a speedup of 80 with 100 processors, what fraction of the original computation can be sequential?

  Answer

  assumption: two modes

  enhanced mode: 100 processors

  serial mode: only 1 processor

Limited Program Parallelism

- Limited parallelism affects speedup
- **Example 1**

  to achieve a speedup of 80 with 100 processors, what fraction of the original computation can be sequential?

  Answer

  by Amdahl's law

$$80 = \frac{1}{\frac{\text{Fraction}_{\text{parallel}}}{100} + (1 - \text{Fraction}_{\text{parallel}})}$$

Limited Program Parallelism

- Limited parallelism affects speedup
- **Example 1**

  to achieve a speedup of 80 with 100 processors, what fraction of the original computation can be sequential?

  Answer

  by Amdahl's law

$$0.8 \times \text{Fraction}_{\text{parallel}} + 80 \times (1 - \text{Fraction}_{\text{parallel}}) = 1$$

$$80 - 79.2 \times \text{Fraction}_{\text{parallel}} = 1$$

$$\text{Fraction}_{\text{parallel}} = \frac{80 - 1}{79.2}$$

$$\text{Fraction}_{\text{parallel}} = 0.9975$$

$$\text{Fraction}_{\text{seq}} = 1 - \text{Fraction}_{\text{parallel}}$$

$$= 0.25\%$$

Limited Program Parallelism

- Limited parallelism affects speedup
- **Example 2**

  app can use 1, 50, or all 100 processors 95% of the time use all 100 processors, how much of the remaining 5% must employ 50 processors for 80 speedup?
  Answer by Amdahl's law

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{100}}{\text{Speedup}_{100}} + \frac{\text{Fraction}_{50}}{\text{Speedup}_{50}} + (1 - \text{Fraction}_{100} - \text{Fraction}_{50})}$$

Limited Program Parallelism

- Limited parallelism affects speedup
- **Example 2**

  app can use 1, 50, or all 100 processors 95% of the time use all 100 processors, how much of the remaining 5% must employ 50 processors for 80 speedup?
  Answer by Amdahl's law: 4.8%

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{100}}{\text{Speedup}_{100}} + \frac{\text{Fraction}_{50}}{\text{Speedup}_{50}} + (1 - \text{Fraction}_{100} - \text{Fraction}_{50})}$$

Limited Program Parallelism

- Limited parallelism in programs makes it difficult to achieve good speedups in any parallel processor;

  in practice, programs often use
  less than the full complement
  of the processors
  when running in parallel mode;

High Communication Cost

- Relatively high cost of communications involves the large latency of remote access in a parallel processor

High Communication Cost

- Relatively high cost of communications involves the large latency of remote access in a parallel processor

  Example

  app running on a 32-processor MP;
  100 ns for reference to a remote mem;
  clock rate 4.0 GHz; base CPI 0.5;
  Q: how much faster if no communication vs if 0.2% remote ref?

High Communication Cost

- **Example**

  app running on a 32-processor MP;

  100 ns for reference to a remote mem;

  clock rate 4.0 GHz; base CPI 0.5;

  Q: how much faster if no communication vs if 0.2% remote ref?

  Answer

  if 0.2% remote reference

$$\begin{aligned} \text{CPI} &= \text{Base CPI} + \text{Remote request rate} \times \text{Remote request cost} \\ &= 0.5 + 0.2\% \times \text{Remote request cost} \end{aligned}$$

High Communication Cost

- **Example**

  app running on a 32-processor MP;

  100 ns for reference to a remote mem;

  clock rate 4.0 GHz; base CPI 0.5;

  Q: how much faster if no communication vs if 0.2% remote ref?

  Answer

  if 0.2% remote ref, Remote req cost

$$\frac{\text{Remote access cost}}{\text{Cycle time}} = \frac{100\text{ns}}{0.25\text{ns}} = 400\,\text{cycles}$$

High Communication Cost

- **Example**

  app running on a 32-processor MP;

  100 ns for reference to a remote mem;

  clock rate 4.0 GHz; base CPI 0.5;

  Q: how much faster if no communicatior
  vs if 0.2% remote ref?

  Answer

  if 0.2% remote ref: 0.5+0.2%x400=1.3

  no comm is 1.3/0.5 = 2.6 times faster

Improve Parallel Processing

- **insufficient parallelism**↓

  new software algorithms that offer better parallel performance;

  software systems that maximize the amount of time spent executing with the full complement of processors;
- **long-latency remote communication**↓

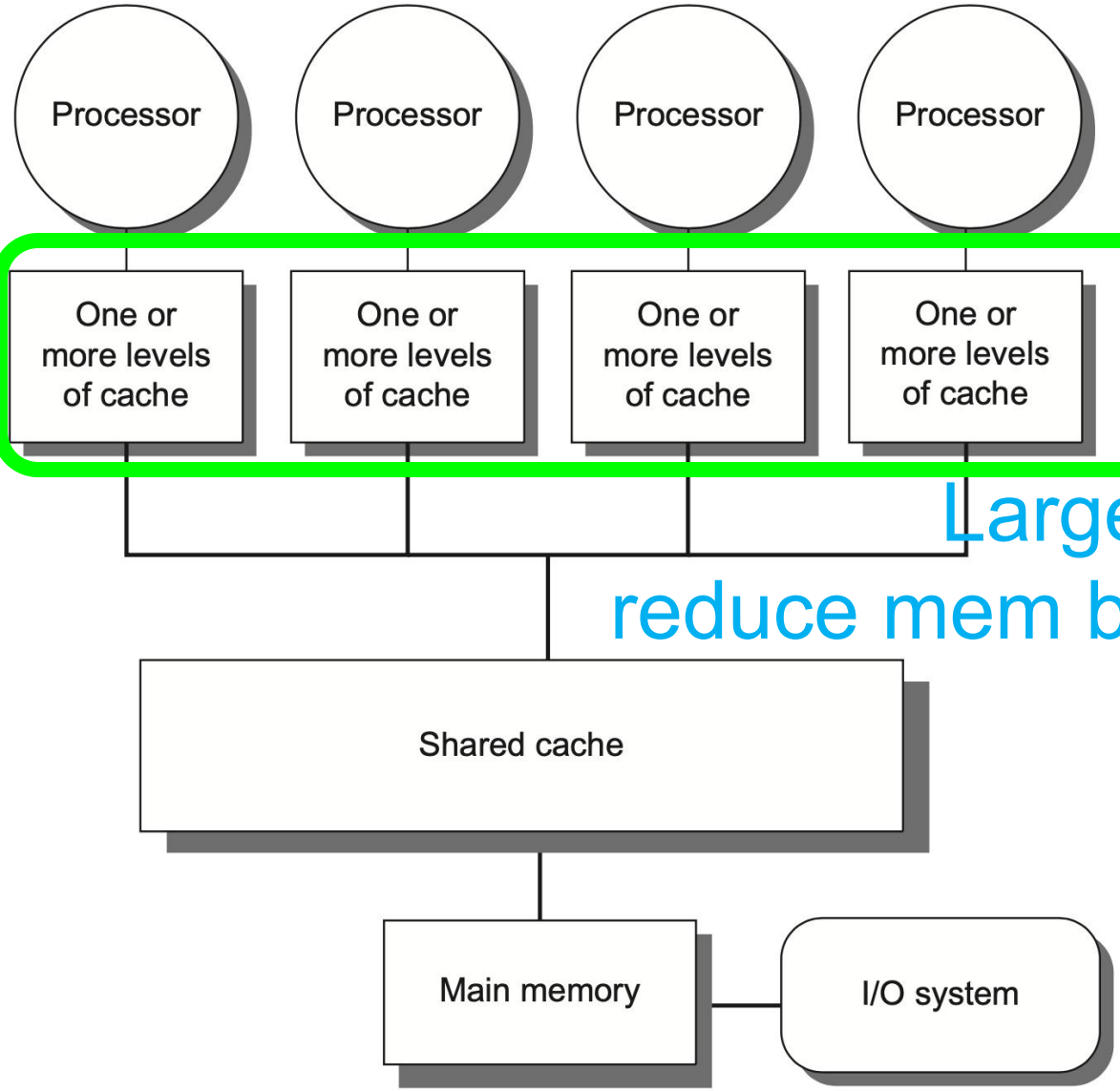
  by architecture: caching shared data…

  by programmer: multithreading, prefetching…

Outline

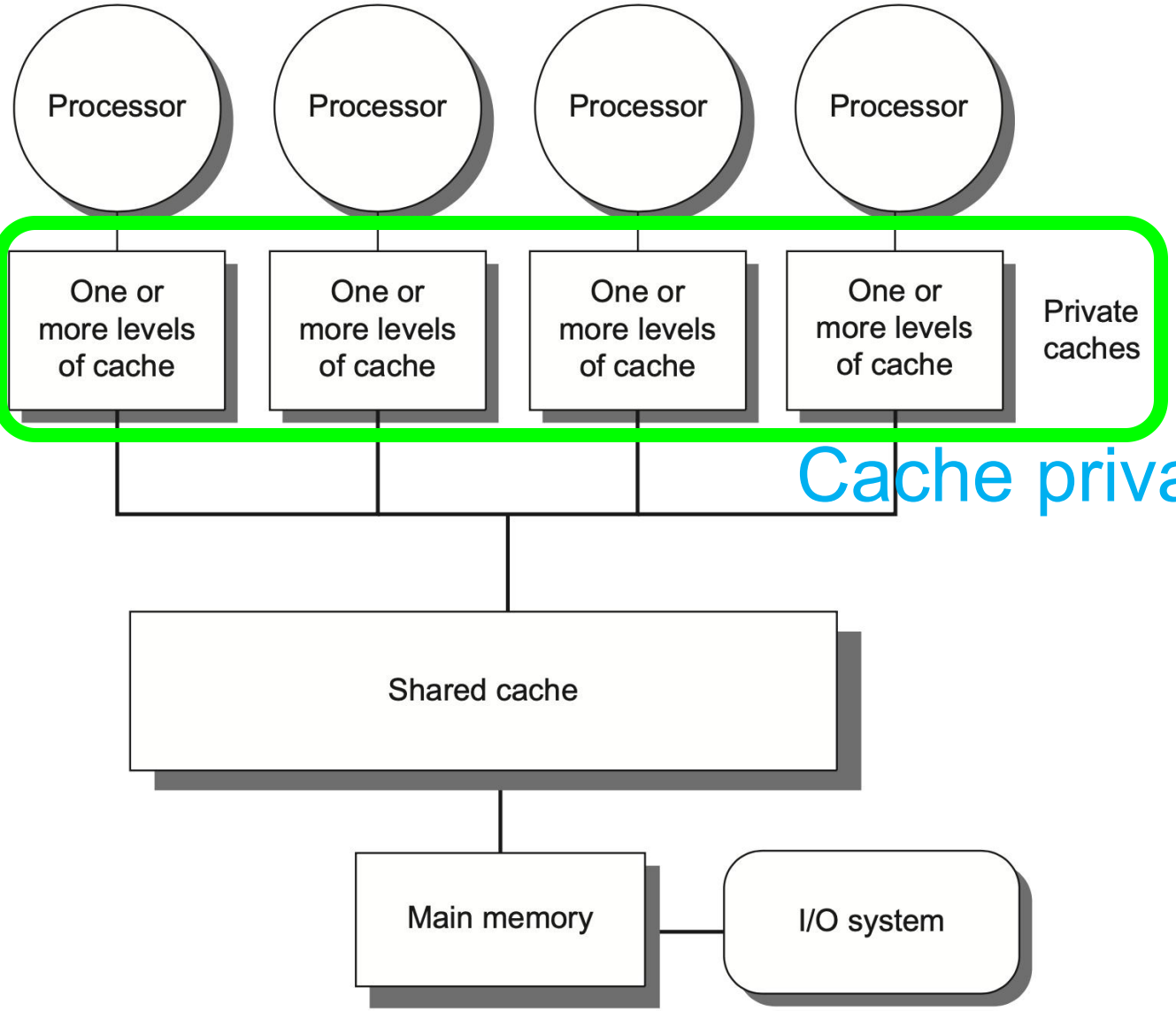
- Multiprocessor Architecture
- Centralized Shared Memory
- Distributed Shared Memory

Centralized Shared-Memory



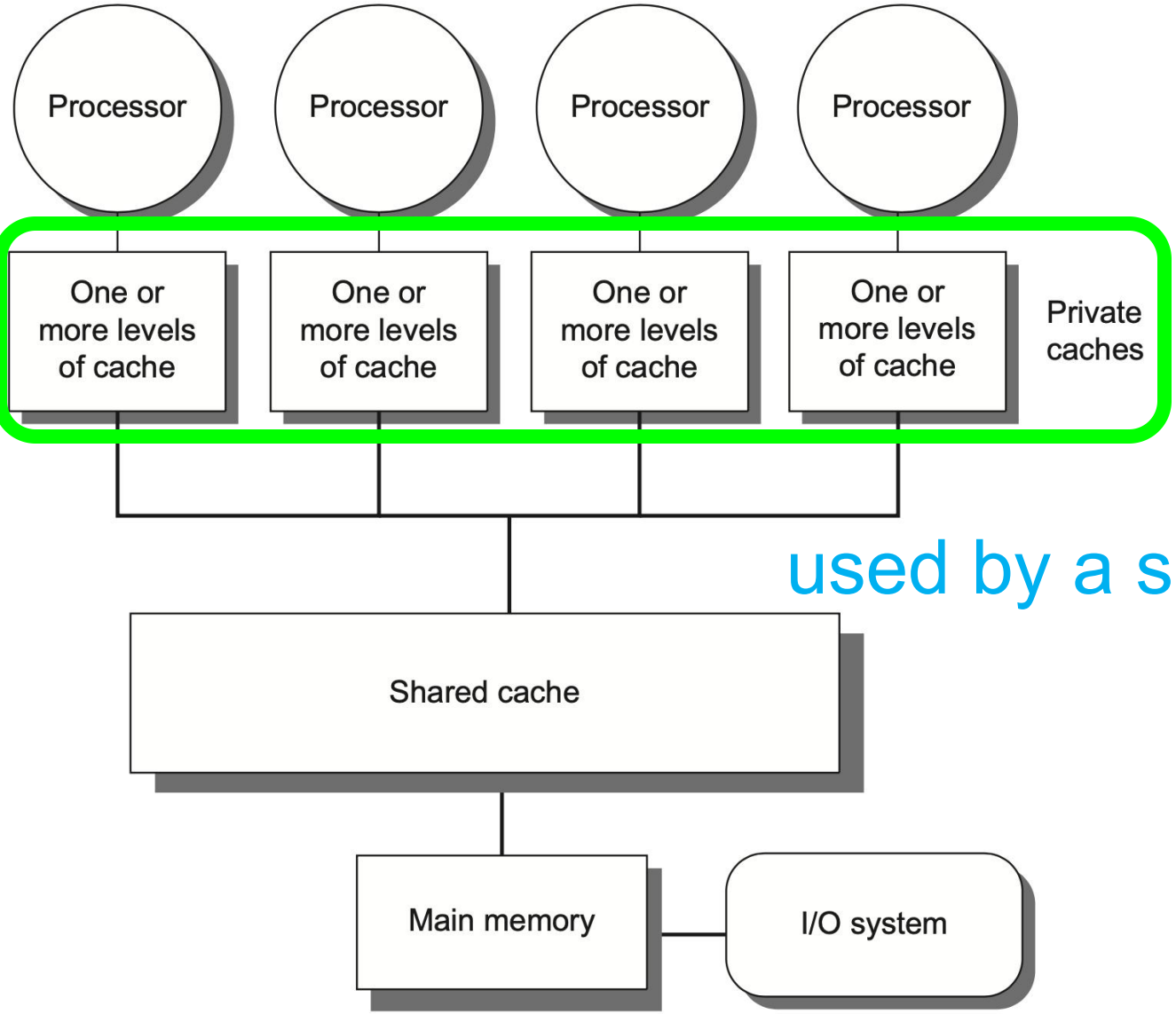
Large, multilevel caches reduce mem bandwidth demands

Centralized Shared-Memory



Cache private/shared data

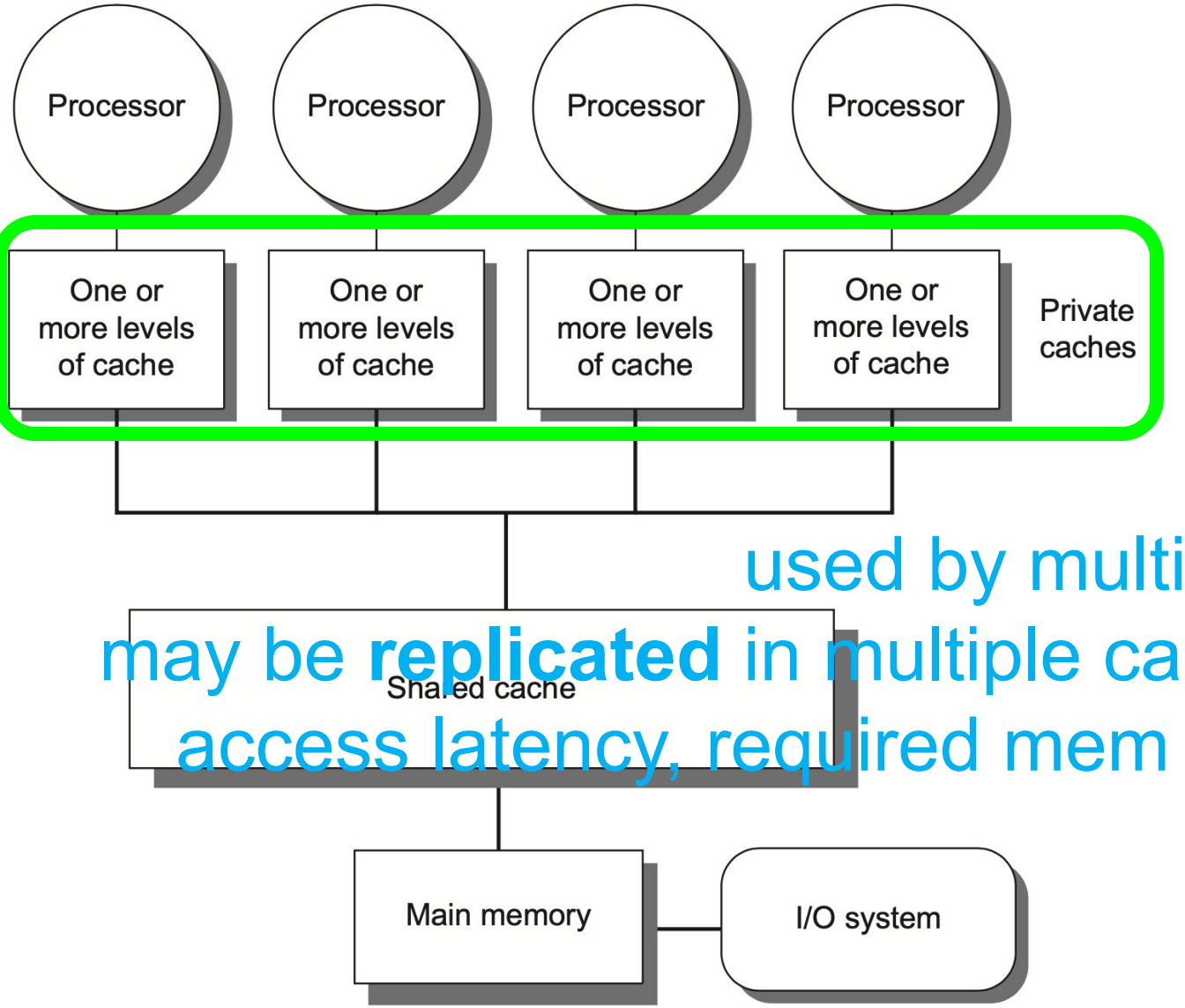
Centralized Shared-Memory



private data

used by a single processor

Centralized Shared-Memory



shared data

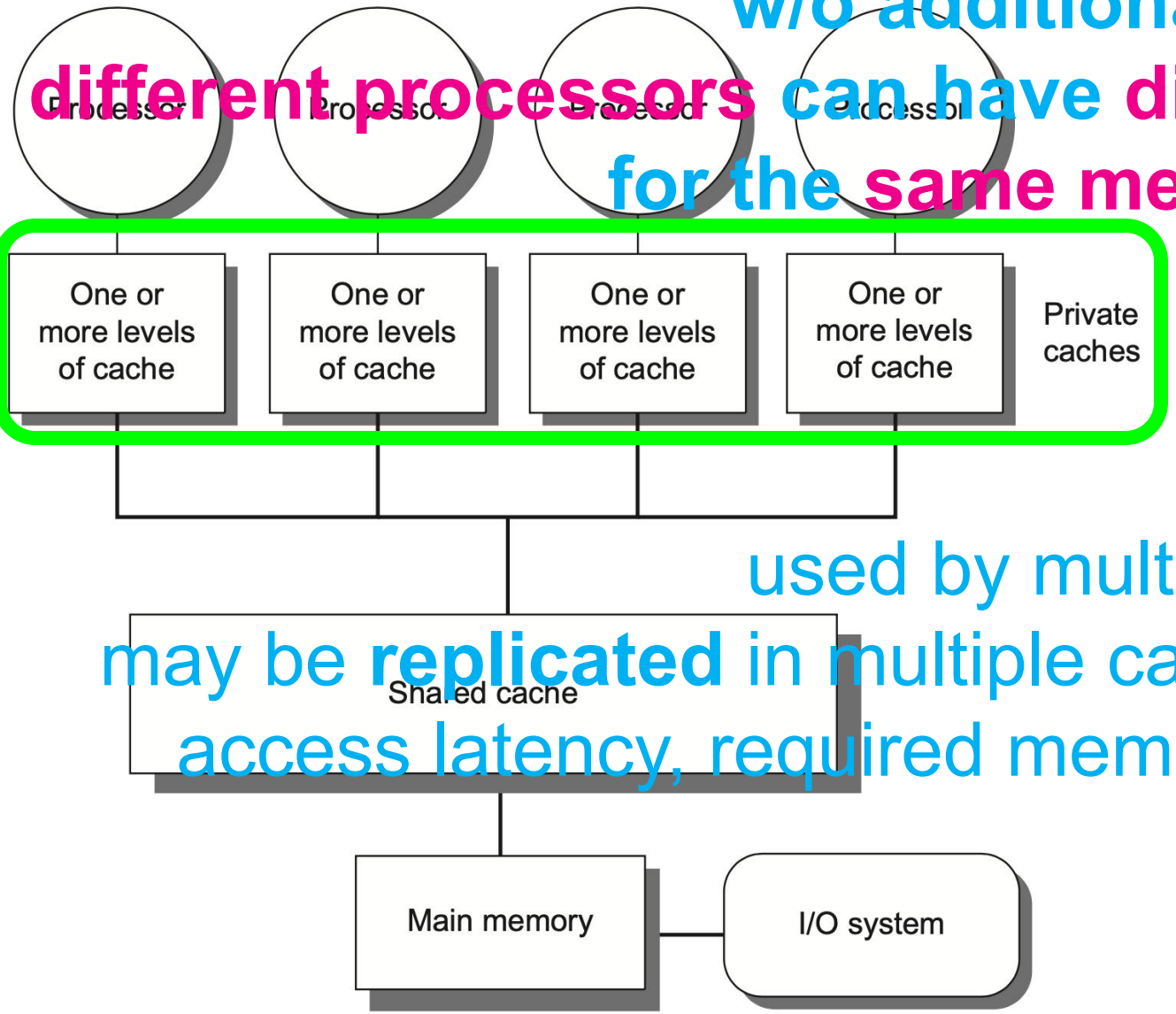
used by multiple processors

may be **replicated** in multiple caches to reduce access latency, required mem bw, contention

Centralized Shared-Memory

w/o additional precautions

different processors can have different values for the same memory location



shared data

used by multiple processors

may be **replicated** in multiple caches to reduce access latency, required mem bw, contention

Cache Coherence Problem

without precautions

Time	Event	Cache contents for processor A	Cache contents for processor B	Memory contents for location X
0				1
1	Processor A reads X	1		1
2	Processor B reads X	1	1	1
3	Processor A stores 0 into X	0	1	0

write-through cache

Cache Coherence Problem

- **Global state** defined by main memory
- **Local state** defined by the individual caches, private to each processor core

Cache Coherence Problem

- A memory system is **Coherent** if any read of a data item returns the most recently written value of that data item
- Two critical aspects

  coherence: defines what values can be returned by a read

  consistency: determines when a written value will be returned by a read

Coherence Property: 1/3

A memory is coherent if: 3-1

- A read by processor P to location X that follows a write by P to X, with no writes of X by another processor occurring between the write and the read by P, always returns the value written by P.

  write -> read: returns written value

  preserves program order

Coherence Property: 2/3

A memory is coherent if: 3-2

- A read by a processor to location X that follows a write by another processor to X returns the written value if the read and the write are sufficiently separated in time and no other writes to X occur between the two accesses.

  write -> read: returns written value

Coherence Property: 3/3

A memory is coherent if: 3-3

- Write serialization

  two writes to the same location by any two processors are seen in the same order by all processors

Consistency

- When a written value will be seen is important
- Memory consistency protocol

Consistency

- Example: a write of X on one processor precedes a read of X on another processor by a very small time, it may be impossible to ensure that the read returns the value of the data written,

  since the written data may not even have left the processor at that point

how to enforce coherence?

Cache Coherence Protocols

- **Directory based**

  the sharing status of a particular block of physical memory is kept in one location, called directory

- **Snooping**

  every cache that has a copy of the data from a block of physical memory could track the sharing status of the block

Snooping Coherence Protocol

- **Write invalidate protocol**

  invalidate other copies on a write

  exclusive access ensures that no other readable or writable copies of an item exist when the write occurs

Snooping Coherence Protocol

- **Write invalidate protocol**
  invalidate other copies on a write

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

write-back cache

update memory when a block becomes shared simplifies the protocol

Snooping Coherence Protocol

- **Write invalidate protocol**
  invalidate other copies on a write

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

write-back cache

shared-state block loses tracking of block ownership

Snooping Coherence Protocol

- **Write invalidate protocol**
  invalidate other copies on a write

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

write-back cache

hard to regulate which core to write-back upon eviction

Snooping Coherence Protocol

- **Write invalidate protocol**
  invalidate other copies on a write

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

write-back cache

update of memory ensures that memory already holds the latest value

Snooping Coherence Protocol

- **Write update/broadcast protocol**

  update all cached copies of a data item when that item is written

  consume more bandwidth

Write Invalidate Protocol

- To perform an invalidate, the processor simply acquires bus access and broadcasts the address to be invalidated on the bus

Write Invalidate Protocol

- To perform an invalidate, the processor simply acquires bus access and broadcasts the address to be invalidated on the bus
- All processors continuously snoop on the bus, watching the addresses

Write Invalidate Protocol

- To perform an invalidate, the processor simply acquires bus access and broadcasts the address to be invalidated on the bus
- All processors continuously snoop on the bus, watching the addresses
- The processors check whether the address on the bus is in their cache;

  if so, the corresponding data block in the cache is invalidated.

Write Invalidate Protocol

- To perform an invalidate, the processor simply acquires bus access and broadcasts the address to be invalidated on the bus
- All processors continuously snoop on the bus, watching the addresses
- If a processor finds that it has a dirty copy of a requested cache block,

  it provides that cache block and causes memory/L3 access to be aborted.

how to implement?

Write Invalidate Protocol

- Incoporate a finite-state controller in each core
- Repspond to requests from the processor in the core and from the bus (or other broadcast medium)
- Change the state of the selected cache block
- Use the bus to access data or to invalidate it

Write Invalidate Protocol

MSI protocol: three block states

- **Invalid**
- **Shared**

  indicates that the block in the private cache is potentially shared
- **Modified**

  indicates that the block has been updated in the private cache;

  implies that the block is **exclusive**

Write Invalidate Protocol

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or *ownership* misses, since they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to share data: place cache block on bus and change state to shared. write-back block
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

Write Invalidate Protocol

replacement

write-back cache

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or *ownership* misses, since they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to share data: place cache block on bus and change state to shared. write-back block
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

Write Invalidate Protocol

figure 5.4

update memory to simplify protocol

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or *ownership* misses, since they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to share data: place cache block on bus and change state to shared. write-back block
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

Write Invalidate Protocol

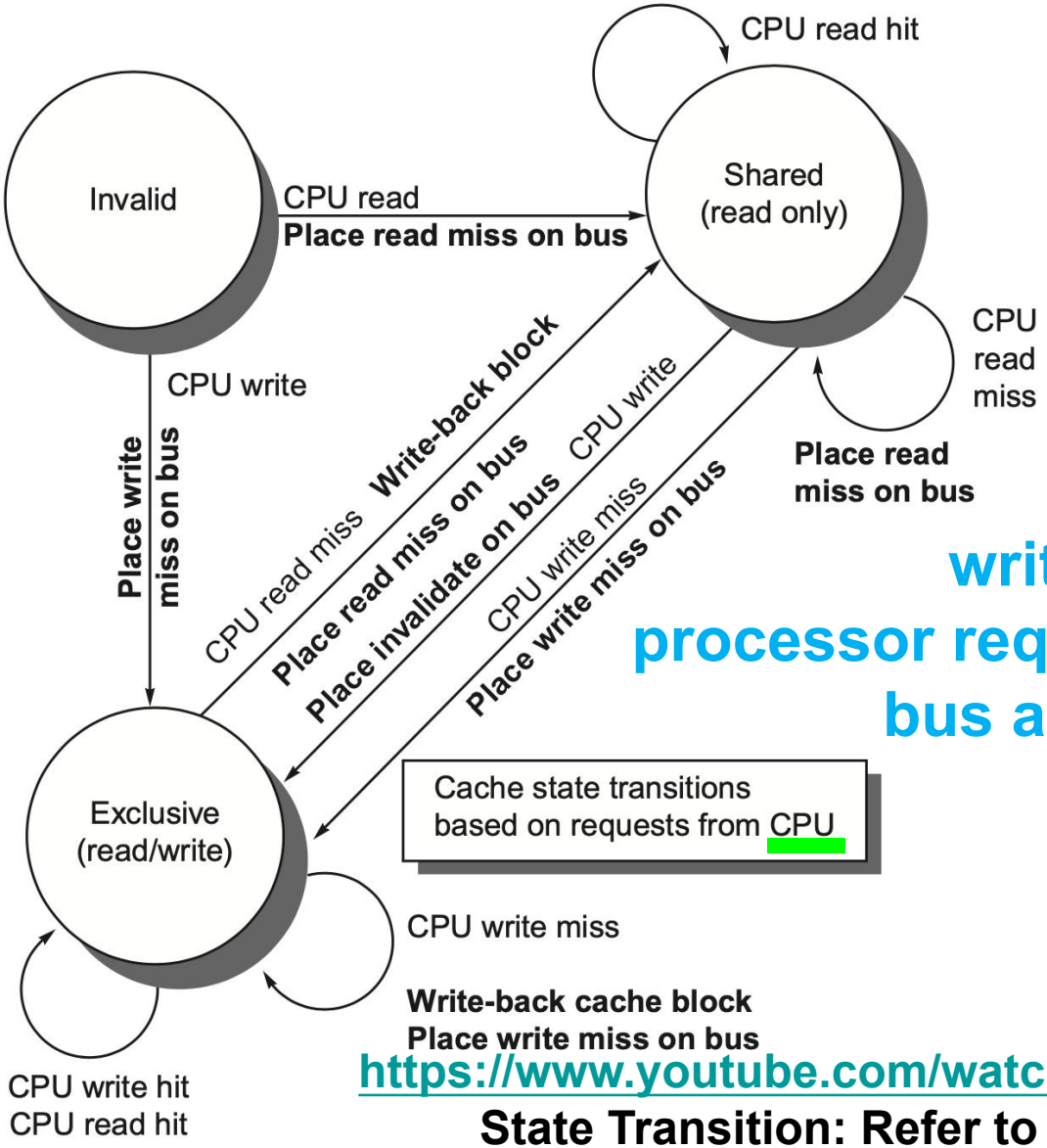
figure 5.6

need also place cache block on bus to service write miss

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or *ownership* misses, since they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to share data: place cache block on bus and change state to shared. write-back block
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

how do states transit?

Write Invalidate Protocol

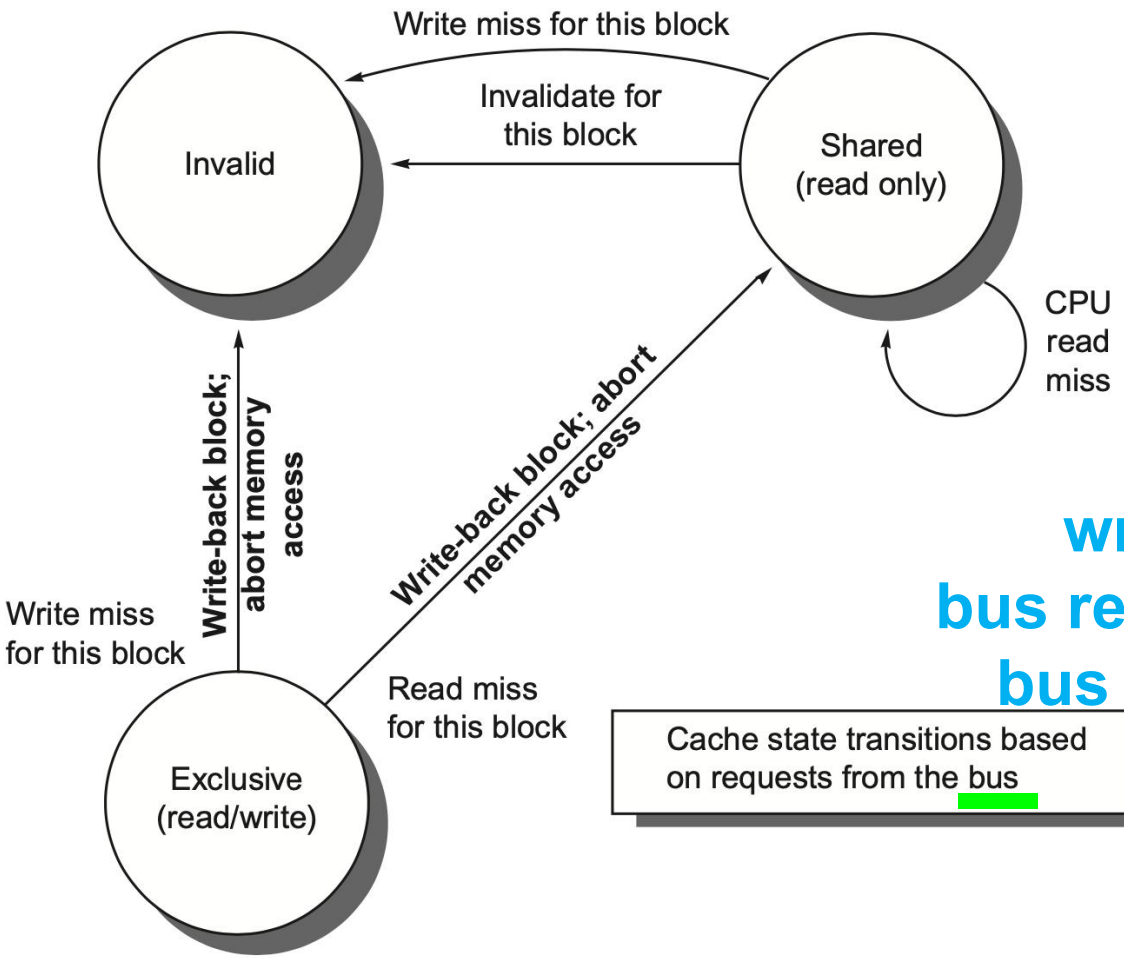


write-back cache
processor requests on arcs
bus actions in bold

https://www.youtube.com/watch?v=gAUVAeI-2Fg

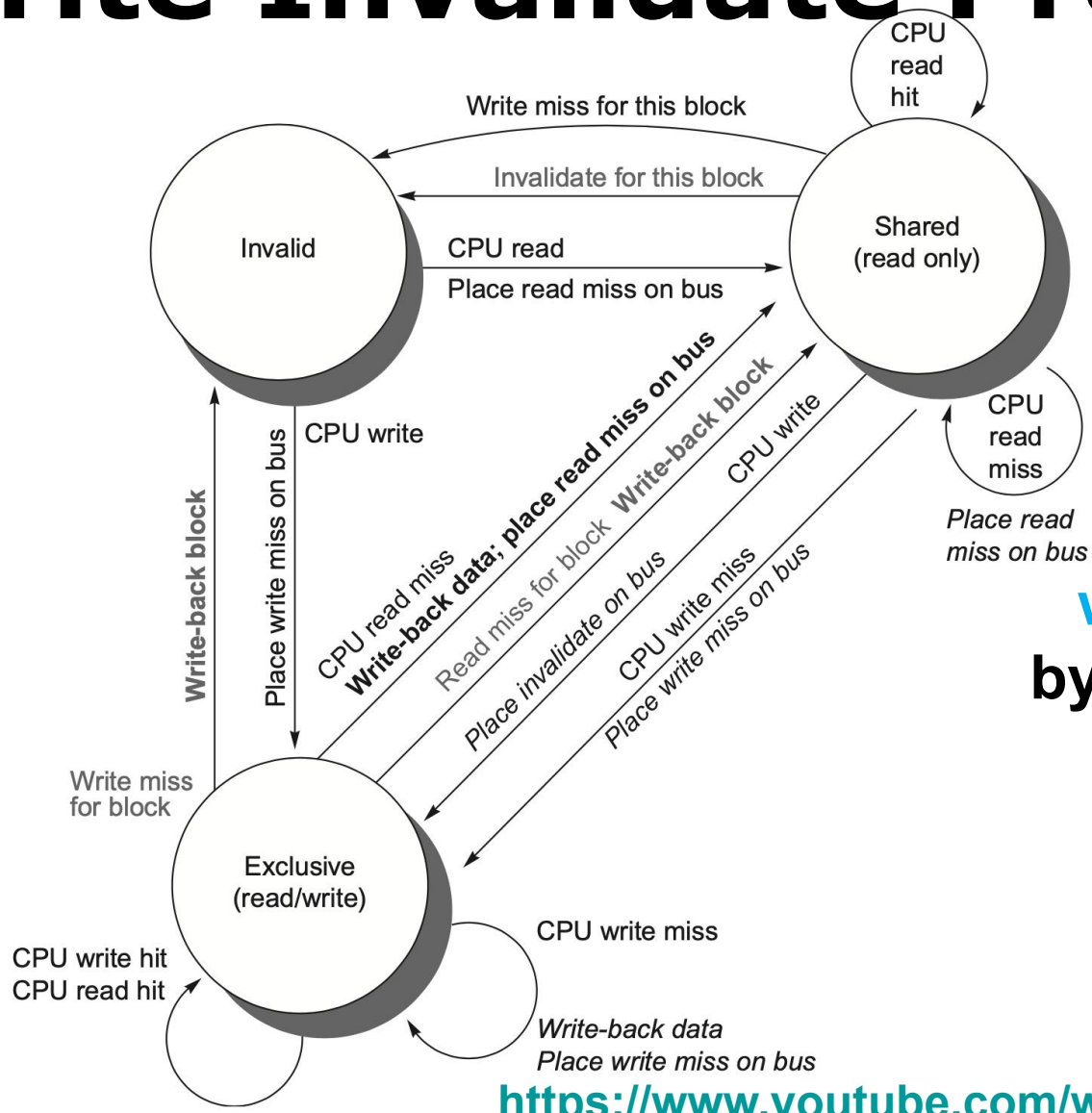
State Transition: Refer to the previous table

Write Invalidate Protocol



write-back cache
bus requests on arcs
bus actions in bold

Write Invalidate Protocol



write-back cache

by local processor

by bus activities

https://www.youtube.com/watch?v=gAUVAeI-2Fg

State Transition: Refer to the previous table

MSI Extensions: MESI

- **MESI**

  exclusive: indicates when a cache block is resident only in a single cache but is clean (after initial read)

  exclusive->read by others->shared;

  exclusive->local write->modified

MSI Extensions: MESI

- **MESI**

  exclusive: indicates when a cache block is resident only in a single cache but is clean (after initial read)

  exclusive->read by others->shared;

  exclusive->local write->modified (without generating any invalidates)

writing an E-block need not invalidate on bus

MSI Extensions: MOESI

- **MOESI**

  owned: indicates that the associated block is owned by that cache and out-of-date in memory

  modified -> owned
  upon a read miss on bus

MSI Extensions: MOESI

- **MOESI**

  owned: indicates that the associated block is owned by that cache and out-of-date in memory

  modified -> owned
  upon a read miss on bus

without writing the shared block to memory

MSI Extensions: MOESI

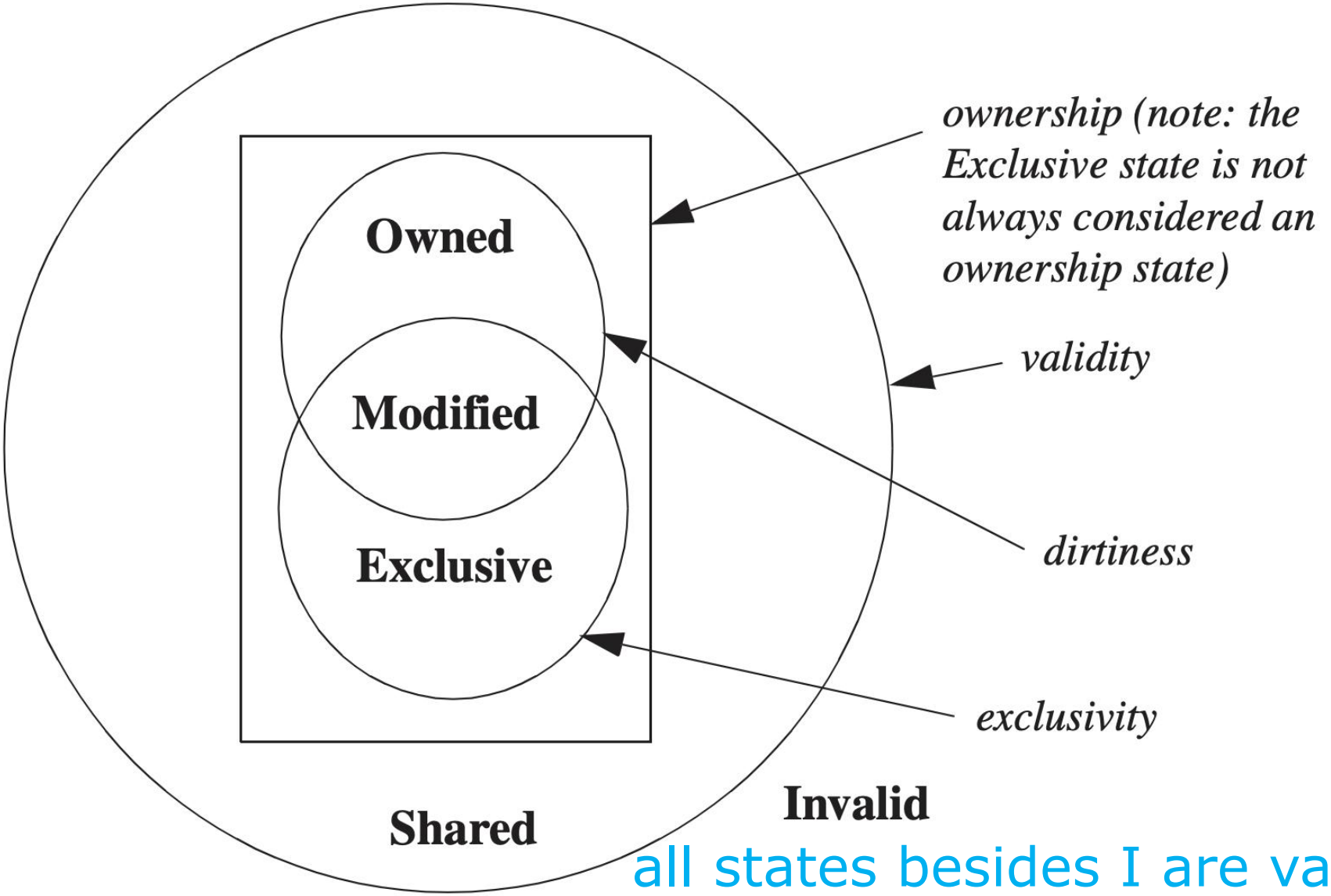
- **MOESI**

  owned: indicates that the associated block is owned by that cache and out-of-date in memory

  modified -> owned
  upon a read miss on bus

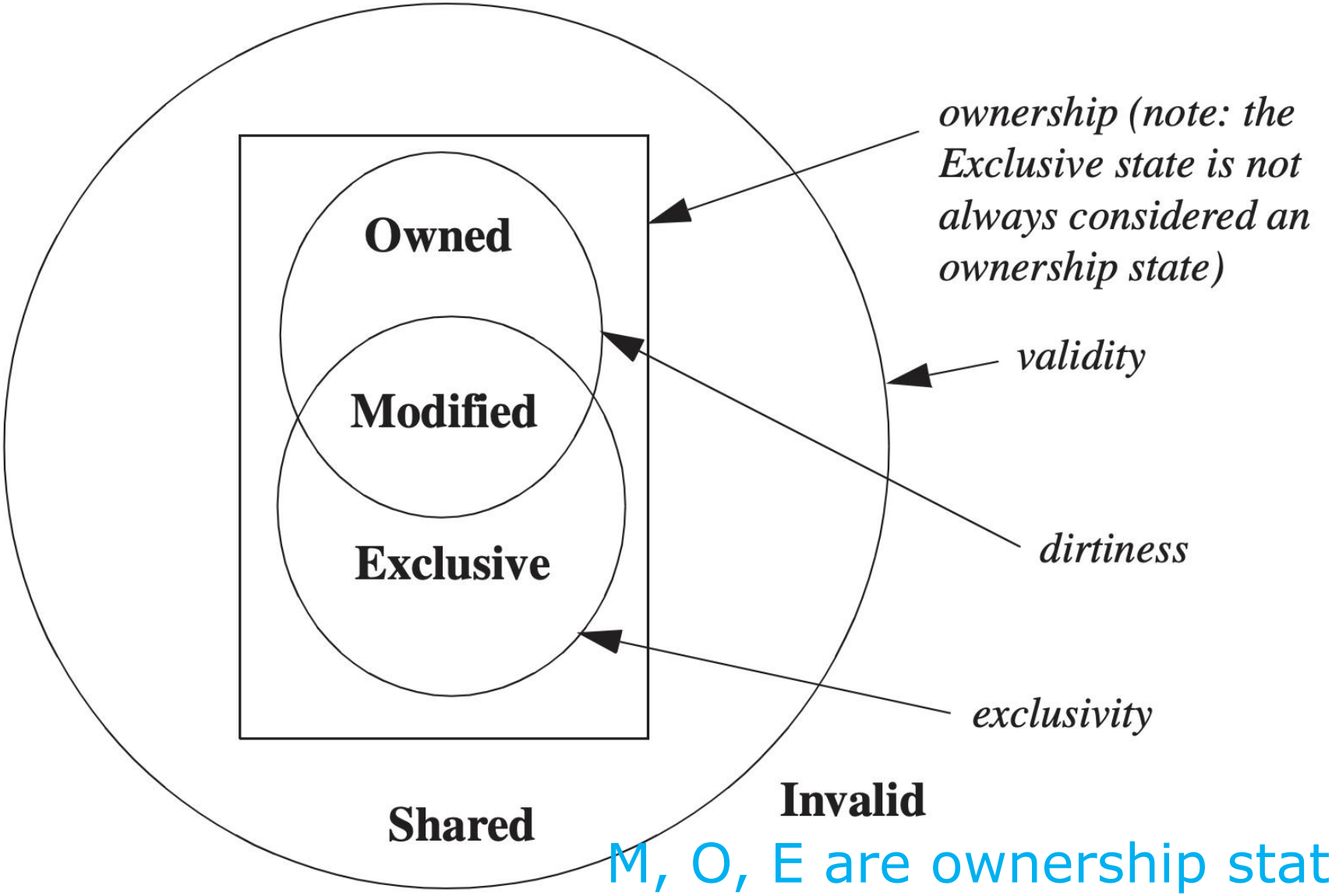
owner writes back to mem upon replacement

MSI Extensions: MOESI



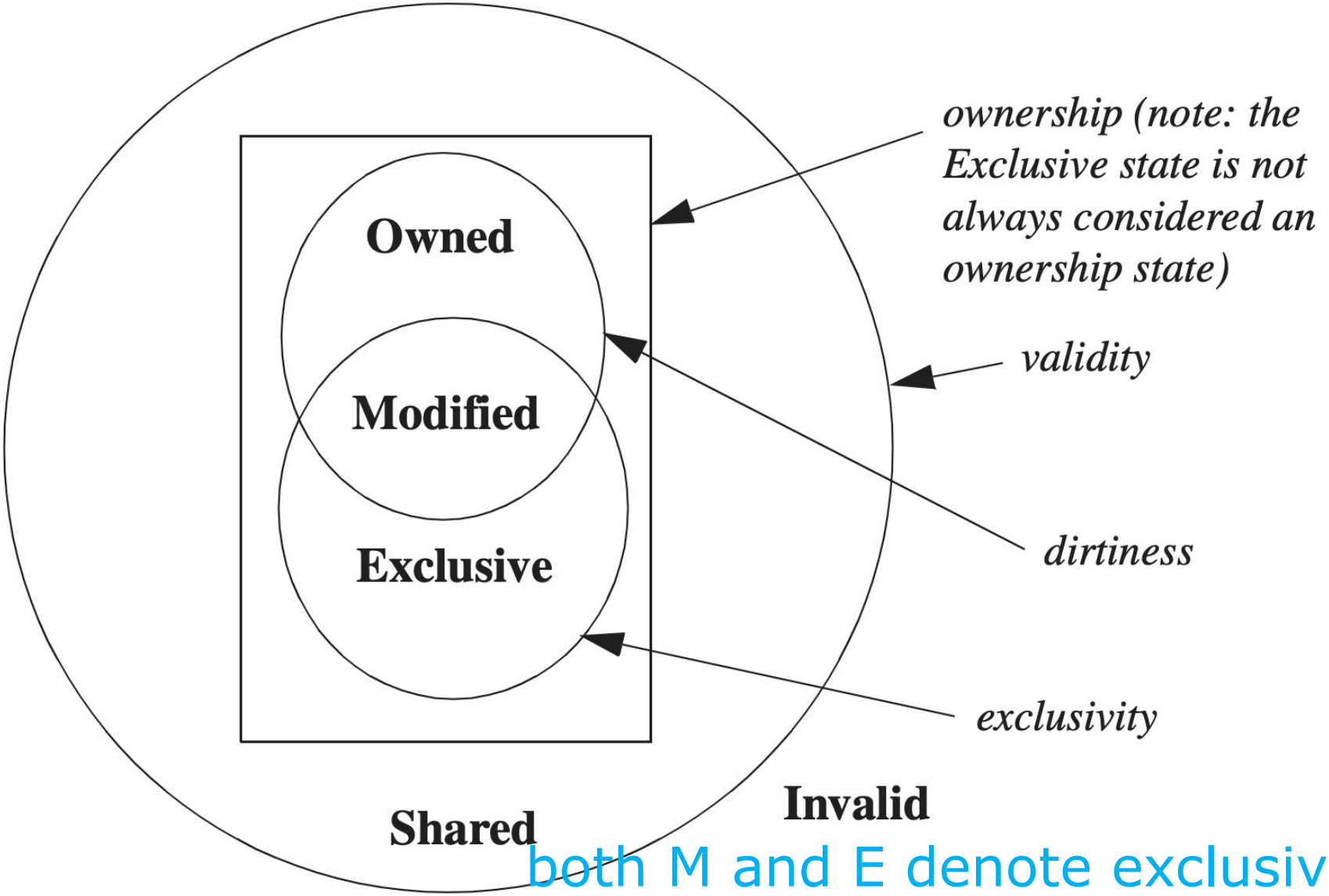
all states besides I are valid

MSI Extensions: MOESI



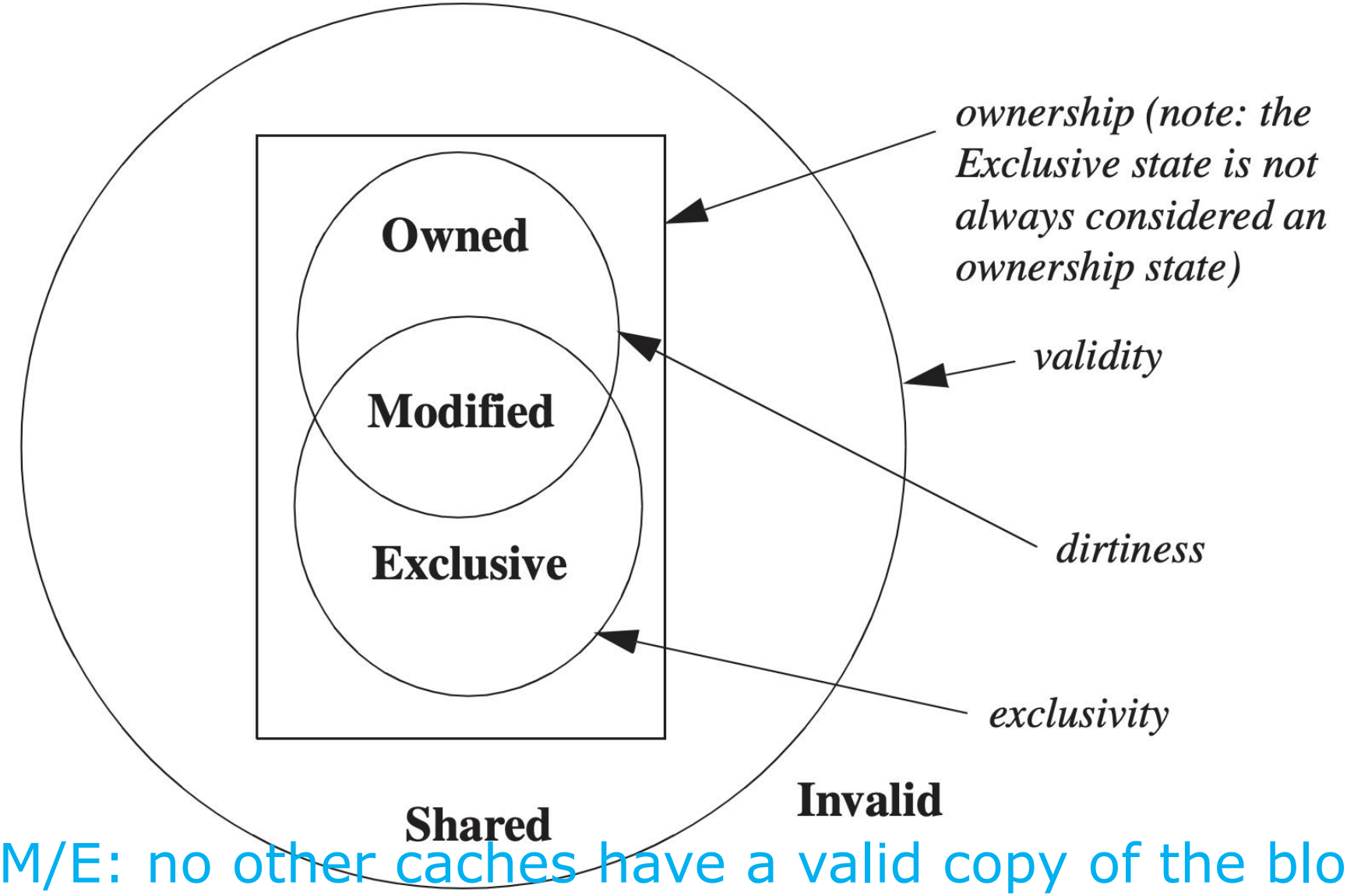
M, O, E are ownership states

MSI Extensions: MOESI



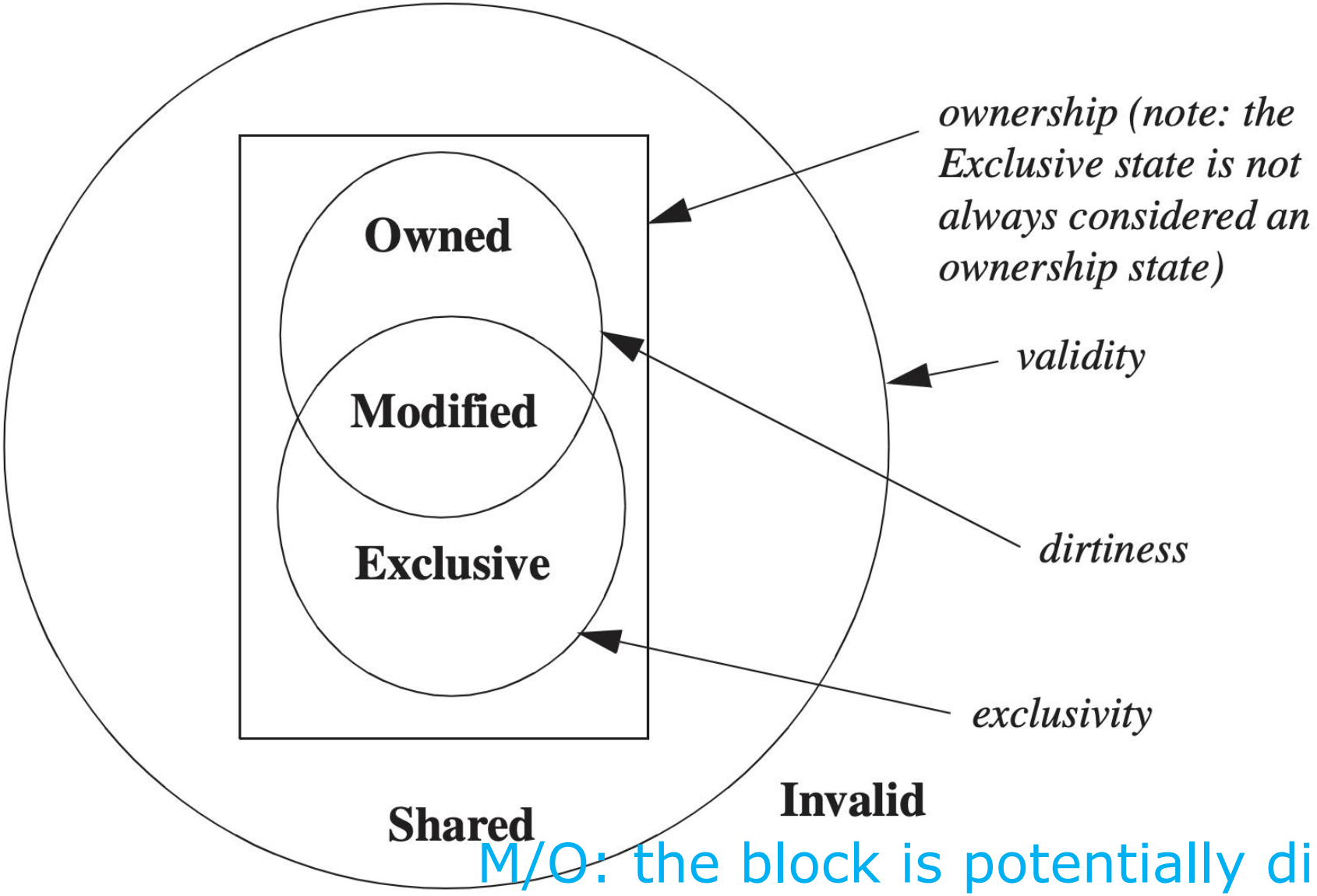
both M and E denote exclusivity

MSI Extensions: MOESI



M/E: no other caches have a valid copy of the block

MSI Extensions: MOESI



M/O: the block is potentially dirty

MESI Example

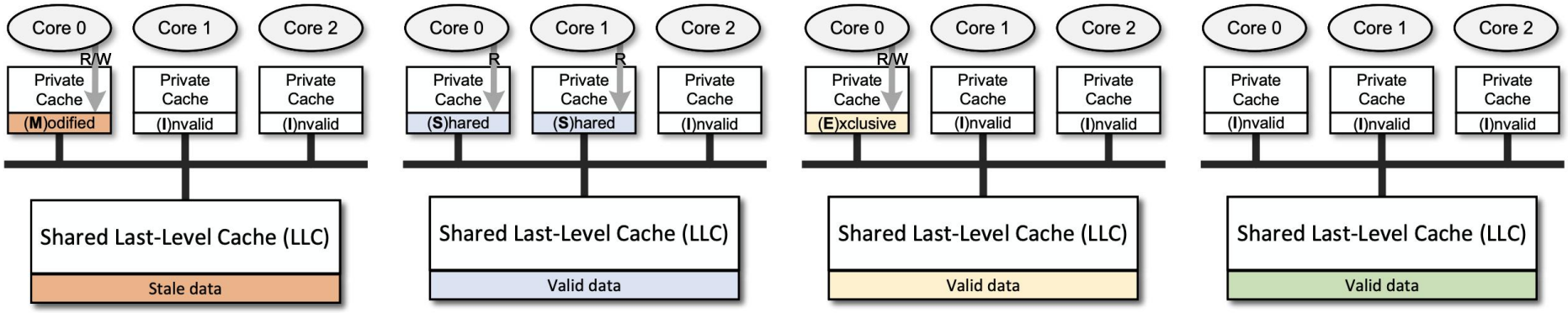


Fig. 1: The four possible states of a private cache line, when using the MESI protocol.

https://www.computer.org/csdl/proceedings-article/sp/2022/131600b550/1FlQFxaLR3G

MESI Example

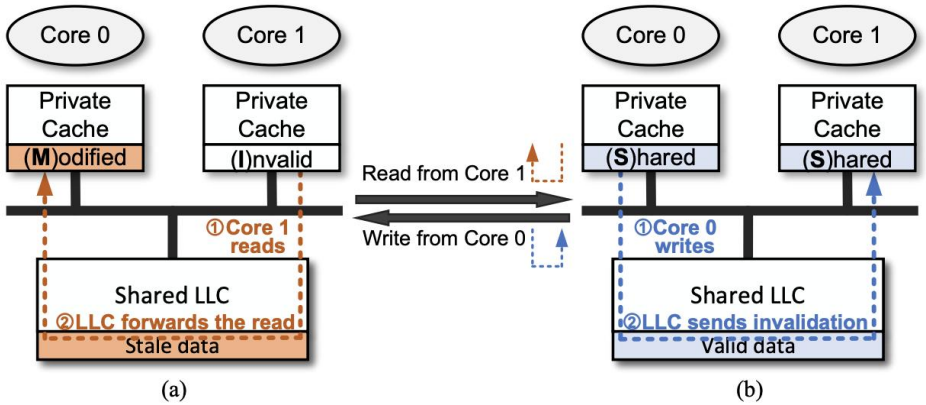


Fig. 2: The illustration of cache coherence state changes. The state of a line changes from M (shown in (a)) to S (shown in (b)) when a CPU core is loading it; conversely, the state changes from S to M when a CPU core is writing it. Dashed lines shows the request path of the read/write operation.

https://www.computer.org/csdl/proceedings-article/sp/2022/131600b550/1FlQFxaLR3G

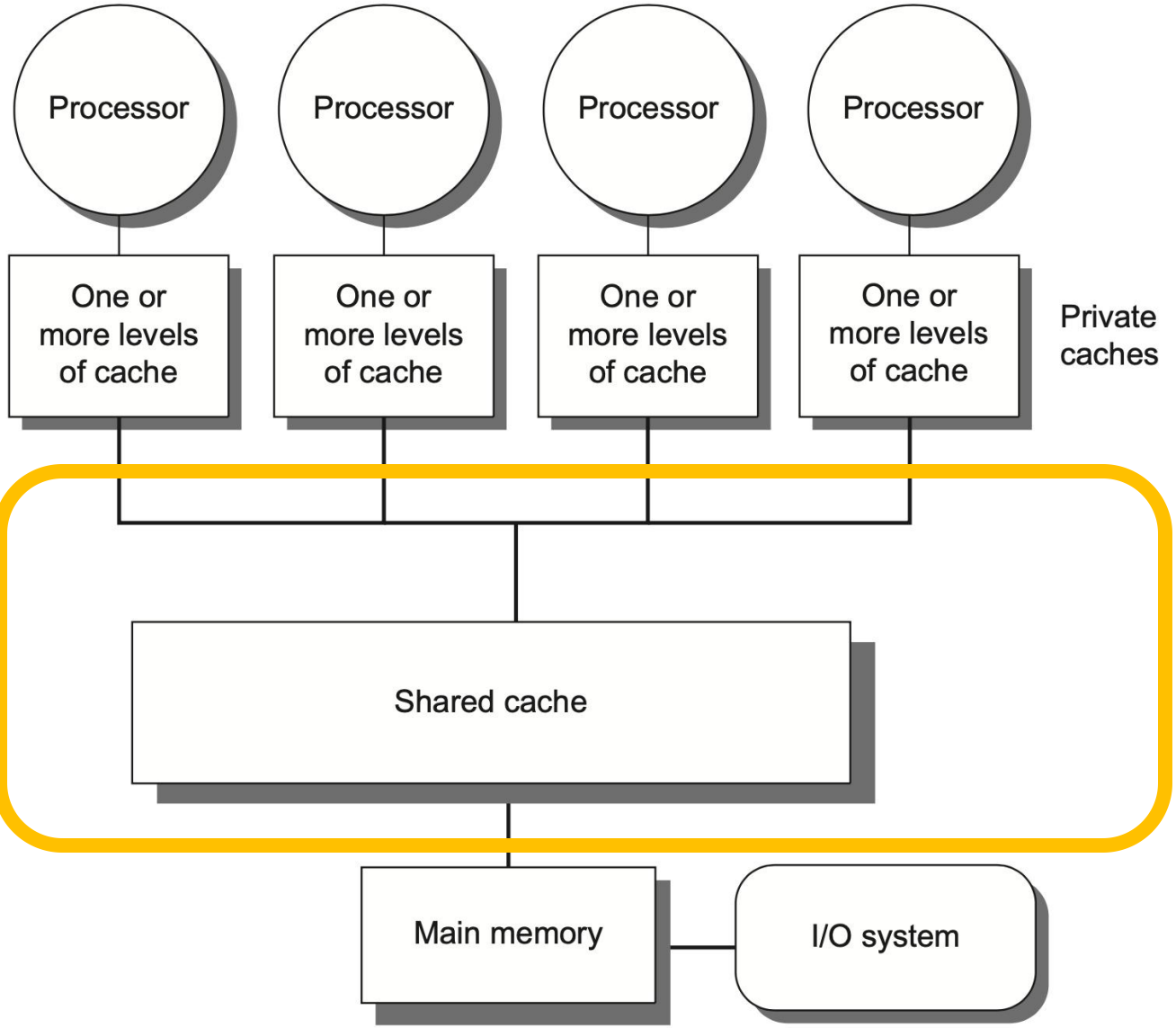
MESIF

- **MESIF**

  forward: designates which sharing processor (among ones with the same shared-state data block) should respond to a request

  the most recent requestor of a line is assigned the F state

Centralized Shared-Memory

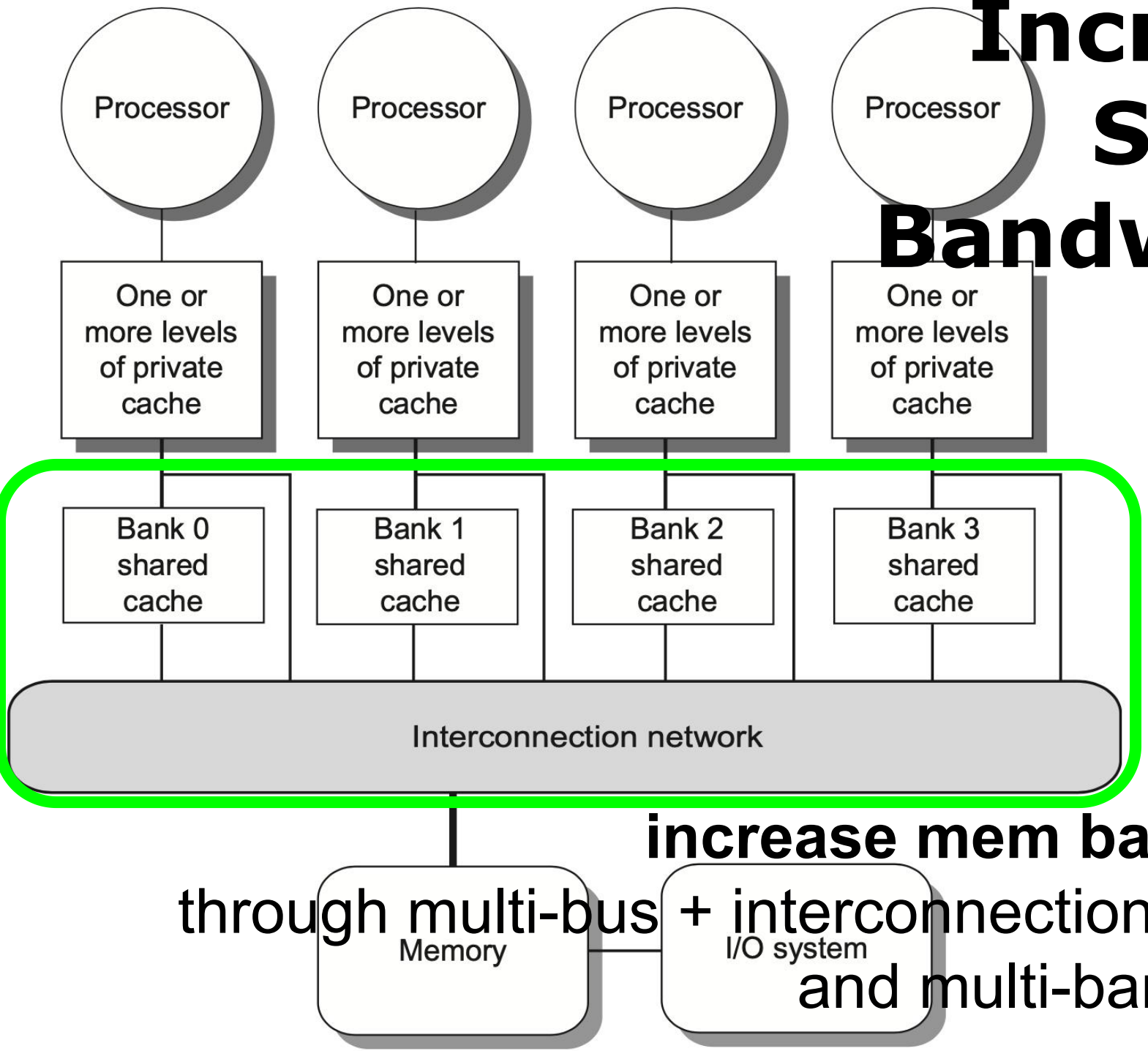


how to de-bottleneck?

Increase Snoop Bandwidth

- Duplicate tags to reduce interference with processor cache accesses
- Distribute outermost shared cache (L3) each processor has a portion of the memory and handles snoops for only that portion of the address space
- Place a directory at outermost shared cache (L3) as a filter on snoop requests that do not associate with cached data

Increase Snoop Bandwidth



increase mem bandwidth

through multi-bus + interconnection network and multi-bank cache

Coherence Miss

- **True sharing miss**

  case 1: first write by a processor to a shared cache block causes an invalidation to establish ownership of that block;

  case 2: another processor reads a modified word in that cache block;

- **False sharing miss**

Coherence Miss

- **True sharing miss**
- **False sharing miss**

  a single valid bit per cache block;

  occurs when a block is invalidated (and a subsequent reference causes a miss) because some word in the block, other than the one being read, is written into

Coherence Miss

- **Example**

  assume words x1 and x2 are in the same cache block, which is in shared state in the caches of both P1 and P2.

  identify each miss as a true sharing miss, a false sharing miss, or a hit?

Time	P1	P2
1	Write x1	
2		Read x2
3	Write x1	
4		Write x2
5	Read x2	

Coherence Miss

- **Example**

Time	P1	P2
1	Write x1	
2		Read x2
3	Write x1	
4		Write x2
5	Read x2	

1. true sharing miss

since x1 was read by P2 and needs to be invalidated from P2

Coherence Miss

- **Example**

Time	P1	P2
1	Write x1	
2		Read x2
3	Write x1	
4		Write x2
5	Read x2	

2. false sharing miss

since x2 was invalidated by the write of x1 in P1,

but that value of x1 is not used in P2;

Coherence Miss

- **Example**

Time	P1	P2
1	Write x1	
2		Read x2
3	Write x1	
4		Write x2
5	Read x2	

3. false sharing miss

since the block is in shared state, need to invalidate it to write;

but P2 read x2 rather than x1;

Coherence Miss

- **Example**

Time	P1	P2
1	Write x1	
2		Read x2
3	Write x1	
4		Write x2
5	Read x2	

4. false sharing miss

need to invalidate the block;

P2 wrote x2 rather than x1;

Coherence Miss

- **Example**

Time	P1	P2
1	Write x1	
2		Read x2
3	Write x1	
4		Write x2
5	Read x2	

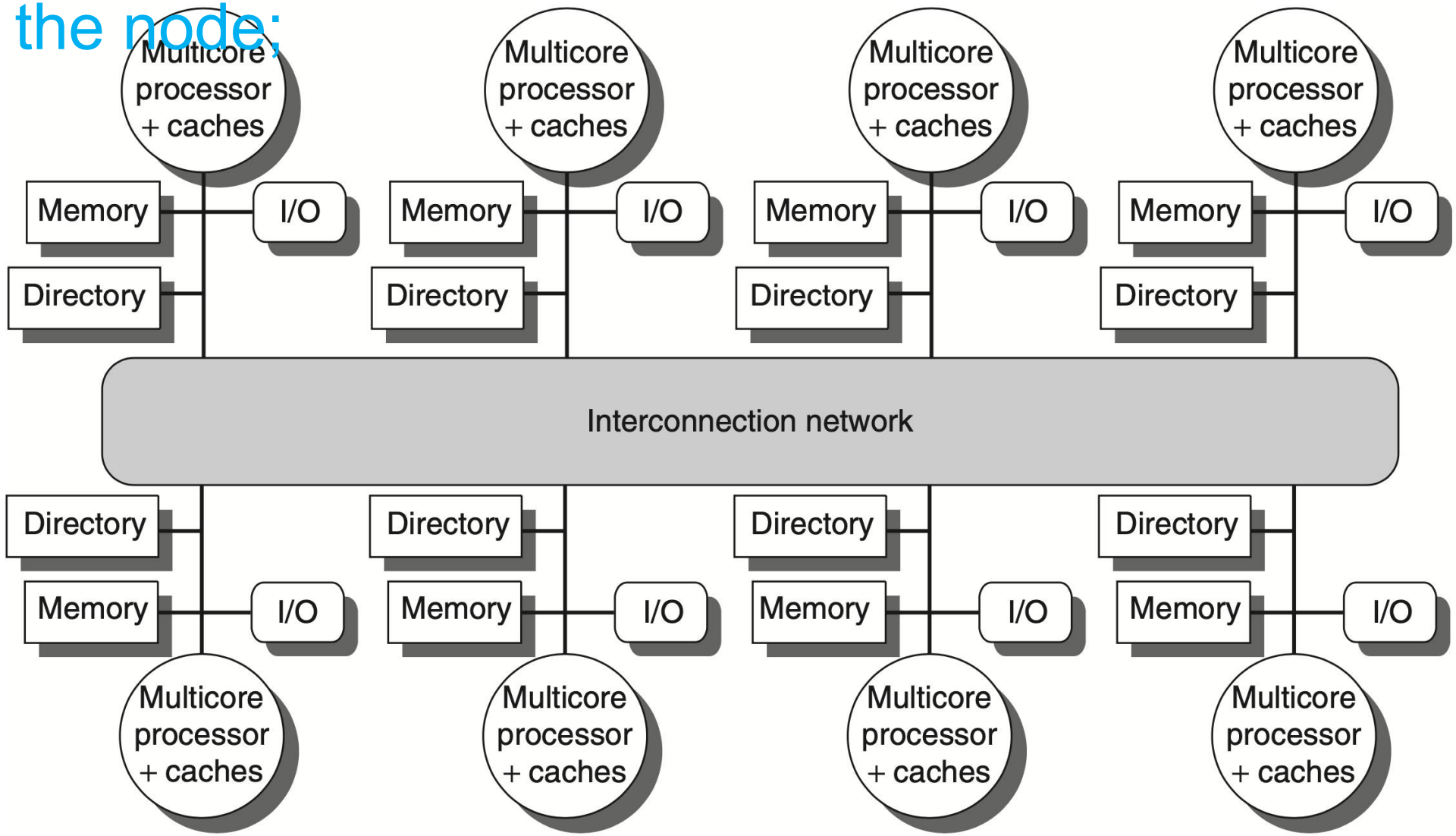
5. true sharing miss

since the value being read was written by P2 (invalid -> shared)

Outline

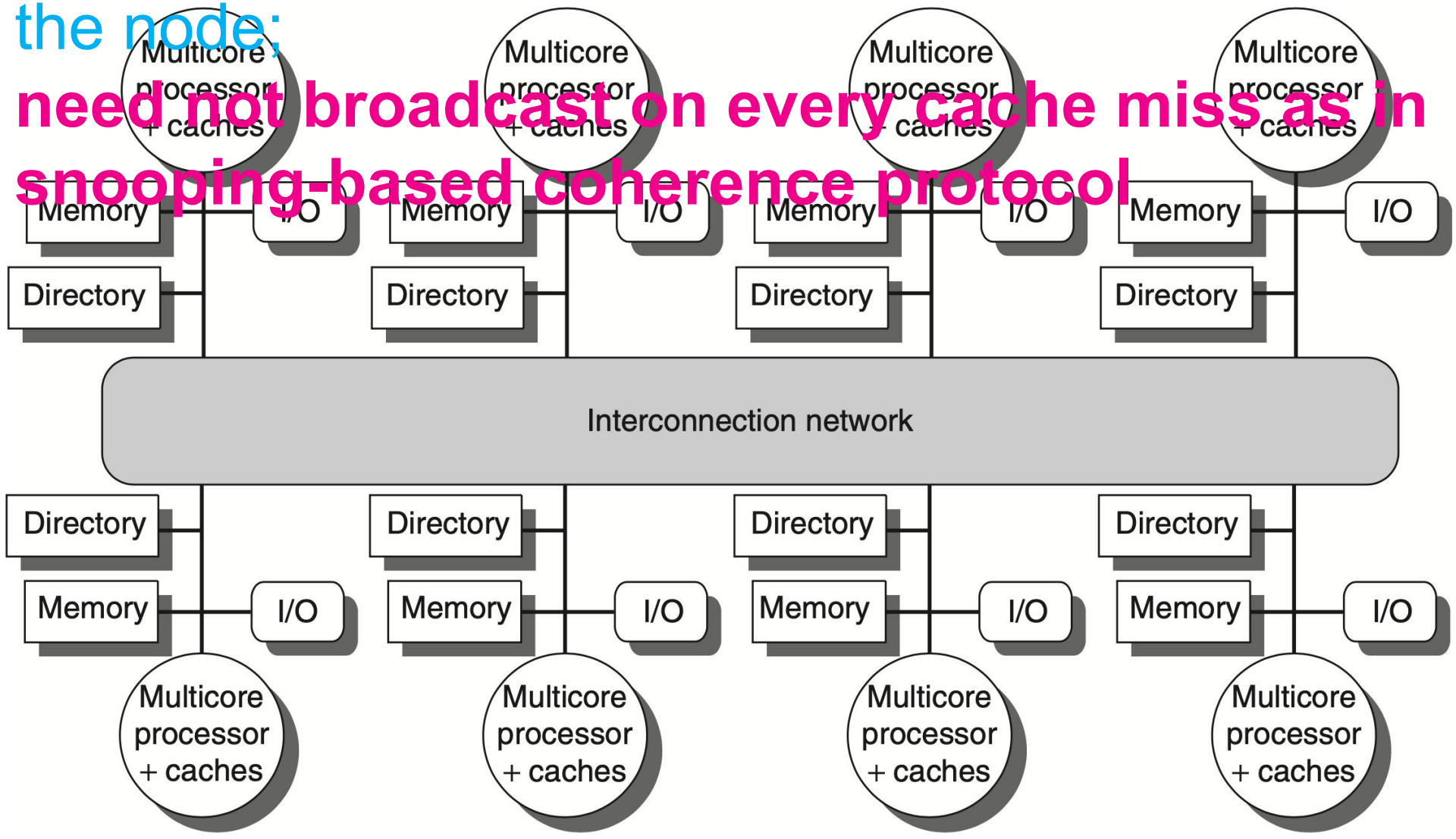
- Multiprocessor Architecture
- Centralized Shared Memory
- Distributed Shared Memory

A **directory** is added to each node;
Each **directory** tracks the caches that share the memory addresses of the portion of memory in the node;



A **directory** is added to each node;
Each **directory** tracks the caches that share the memory addresses of the portion of memory in the node;

need not broadcast on every cache miss as in snooping-based coherence protocol



Directory-based Cache Coherence Protocol

Common cache states

- **Shared**

  one or more nodes have the block cached, and the value in memory is up to date (as well as in all the caches)

- **Uncached**

  no node has a copy of the cache block

- **Modified**

  exactly one node has a copy of the cache block, and it has written the block, so the memory copy is out of date

Directory-based Cache Coherence Protocol

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

P: requesting node number A: requested address D: data contents

Directory-based Cache Coherence Protocol

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

P: requesting node number A: requested address D: data contents

Directory-based Cache Coherence Protocol

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

P: requesting node number A: requested address D: data contents

Directory-based Cache Coherence Protocol

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

P: requesting node number A: requested address D: data contents

Directory-based Cache Coherence Protocol

home node reads a block invalidated by a remote node

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

P: requesting node number A: requested address D: data contents

Directory-based Cache Coherence Protocol

home node writes and invalidates a block invalidated by a remote node

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

P: requesting node number A: requested address D: data contents

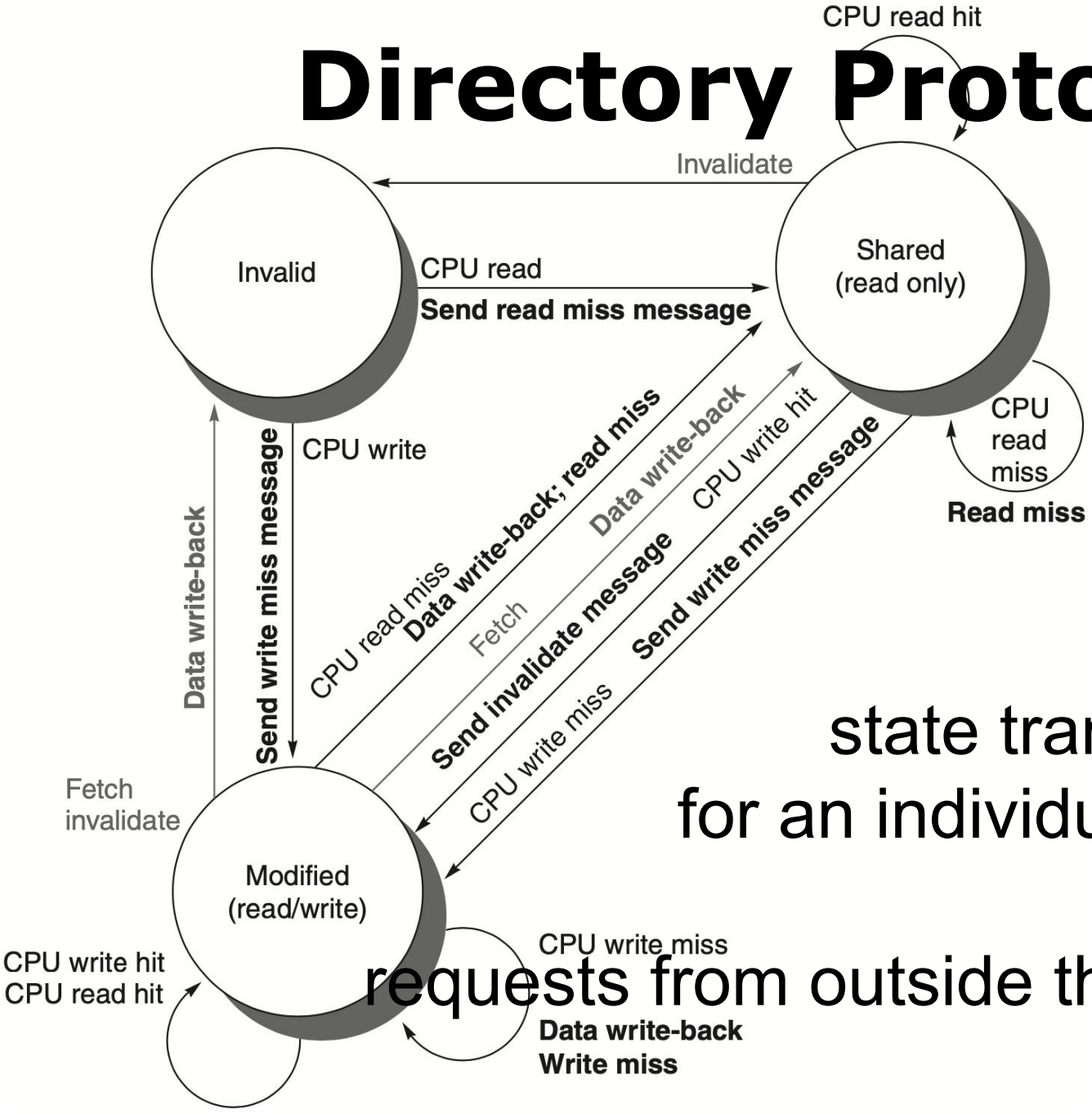
Directory-based Cache Coherence Protocol

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

P: requesting node number A: requested address D: data contents

how do states transit?

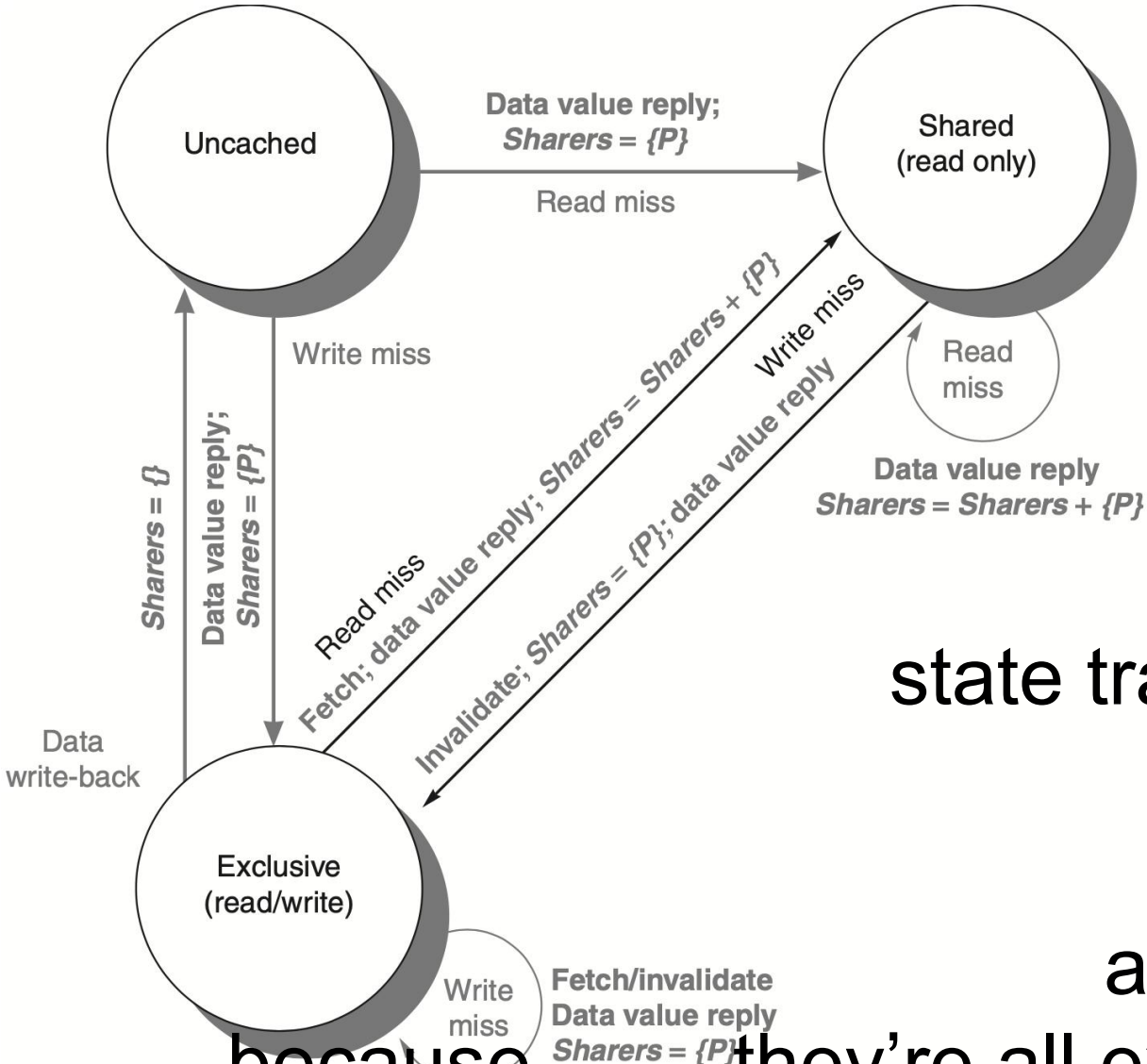
Directory Protocol



state transition diagram
for an individual cache block

requests from outside the node in gray

Directory Protocol



state transition diagram for the directory

all actions in gray because they're all externally caused

Review

- Mutiprocessor architecture
- Cache coherence
- Write invalidate protocol
- Write update/broadcast protocol
- MSI/MESI/MOESI protocol
- True/false sharing miss
- Directory-based cache coherence protocol